第四章 利用机器学习模型进行数据分类

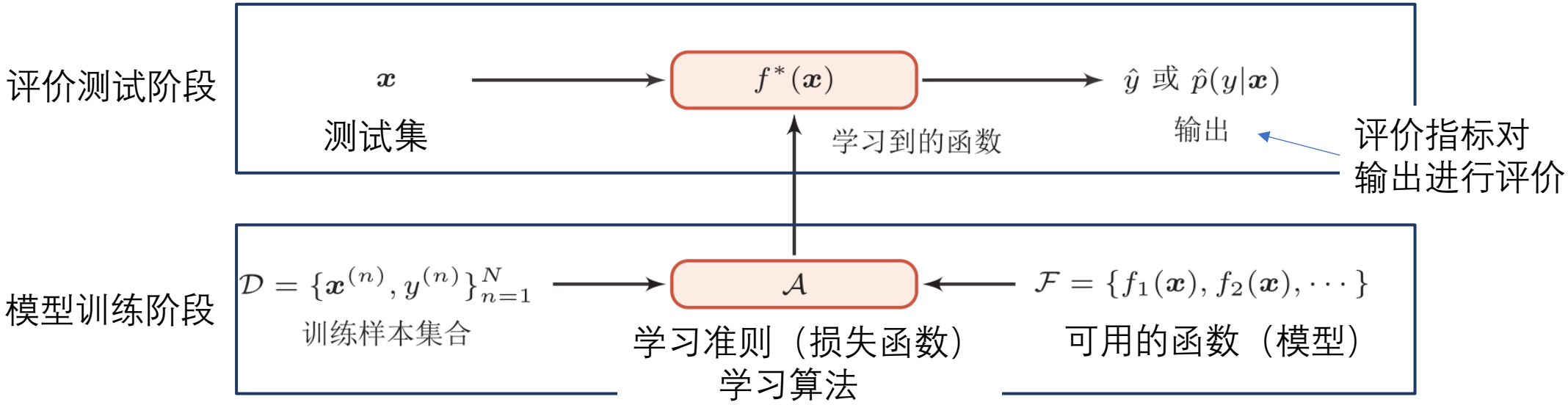
Lianghai Xiao

https://github.com/styluck/mlb

作业邮箱：alswfx@126.com

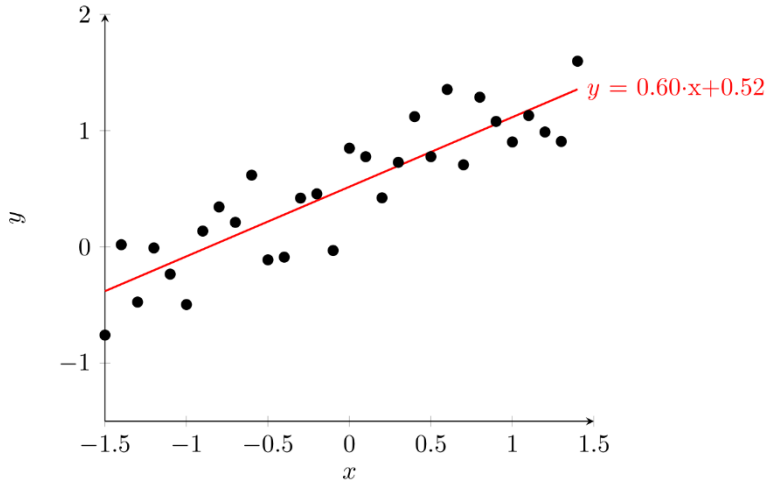
什么是机器学习？

- 机器学习：通过算法使得机器能从大量数据中学习规律从而对新的样本做决策。
  - 规律：决策（预测）函数



模型

- 以线性回归（Linear Reqression）为例
- 模型： $f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^{\mathrm{T}}\mathbf{x} + b$



模型的训练

- 经验风险最小化（最小二乘法）
- 结构风险最小化（岭回归）
- 最大似然估计
- 最大后验估计

- 平方损失函数

$$\mathcal{L}(y, \hat{y}) = (y - f(x, \theta))^2$$

学习准则

- 训练数据：$\mathcal{D} = \{\mathrm{x}^{(\mathrm{n})}, y^{(\mathrm{n})}\}, i \in [1, N]$，经验风险为

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

- 在选择合适的风险函数后，我们寻找一个参数θ^*，使得经验风险函数最小化。

$$\theta^* = \arg\min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

- 机器学习问题转化成为一个最优化问题

线性模型

线性模型

- 线性模型（Linear Model）
  - 是机器学习中应用最广泛的模型，指通过样本特征的线性组合来进行预测的模型．
  - 线性判别函数：

$$
\begin{aligned}
f(\boldsymbol{x};\boldsymbol{w}) &= w_1x_1 + w_2x_2 + \cdots + w_Dx_D + b \\
&= \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b,
\end{aligned}
$$

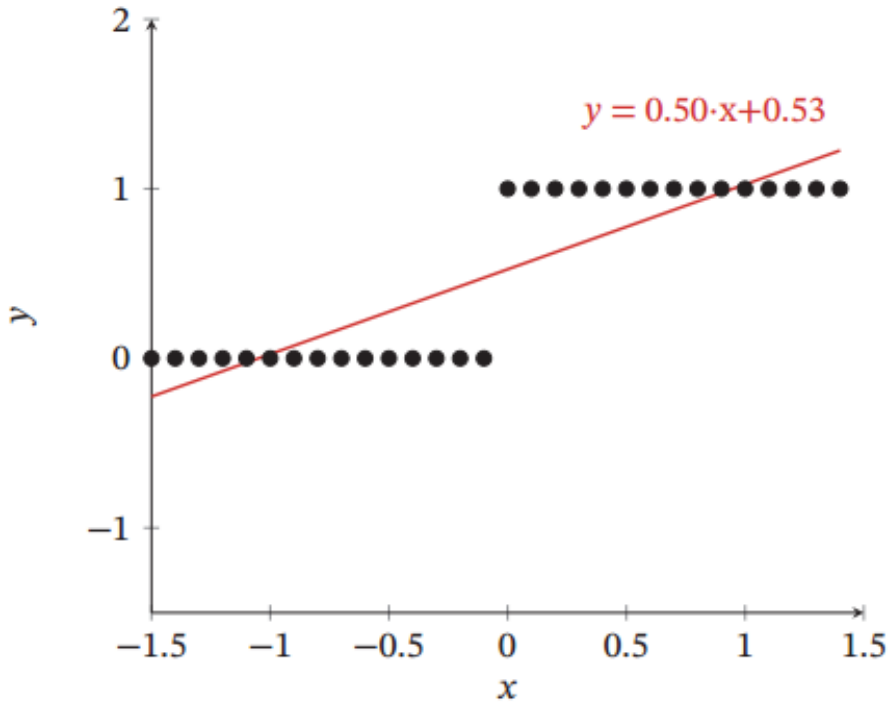
  - 线性的决策函数：

$$
y = f(\boldsymbol{x};\boldsymbol{w})
$$

线性决策函数

- 如果y是连续值，线性决策函数效果较好。但线性的决策函数预测分类问题的输出效果差，因为输出目标y是一些离散的标签

$$y = f(\boldsymbol{x}; \boldsymbol{w})$$

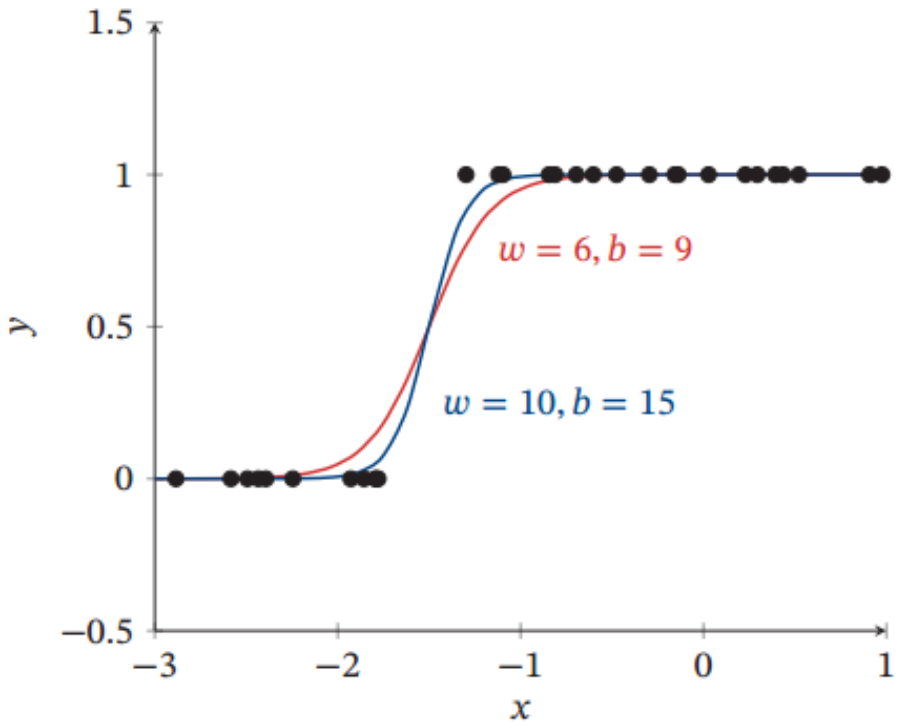


非线性决策函数

- 引入一个非线性的决策函数来更好的进行拟合

$$y = g(f(\boldsymbol{x}; \boldsymbol{w}))$$

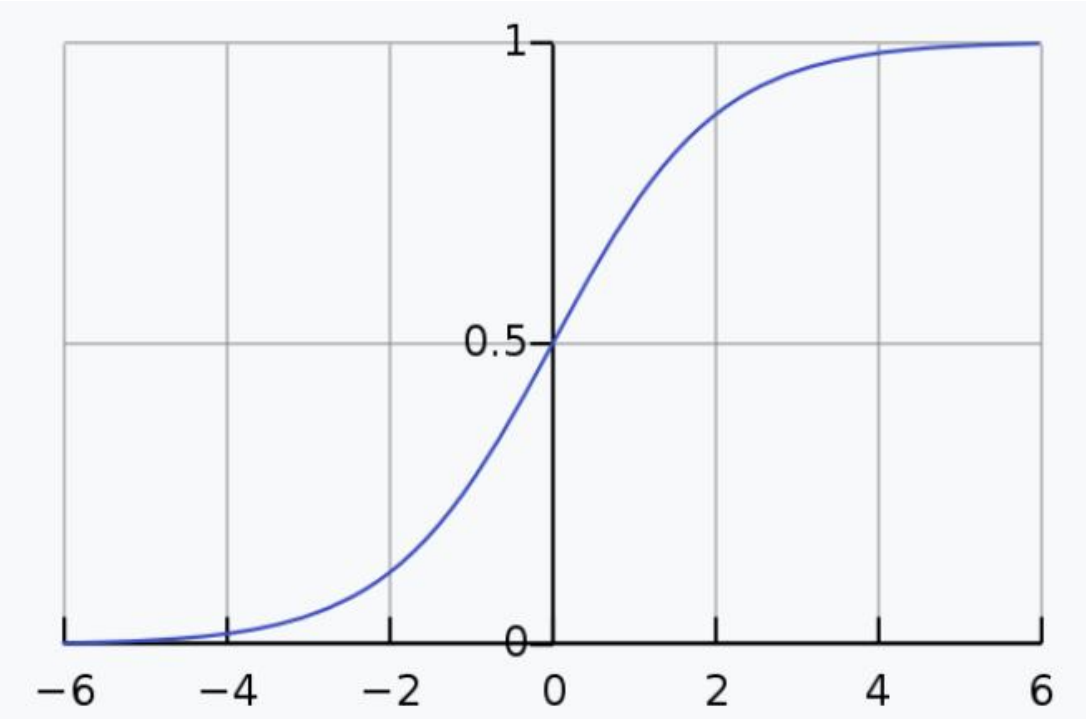
- 把线性函数的值域从实数区间“挤压”到了(0,1)之间，可以用来表示概率。



Logistic函数与回归

- Logistic函数(Sigmoid函数)

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



- 优点
  - 将任意实数映射到 [0,1] 范围内
  - 任意阶可导
  - 在0 附近几乎是线性的，但在两端趋于平缓
  - 将异常值压向 0 或 1

其他常用的激活函数

- Hyperbolic tangent

$$f(x) = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Arctangent function

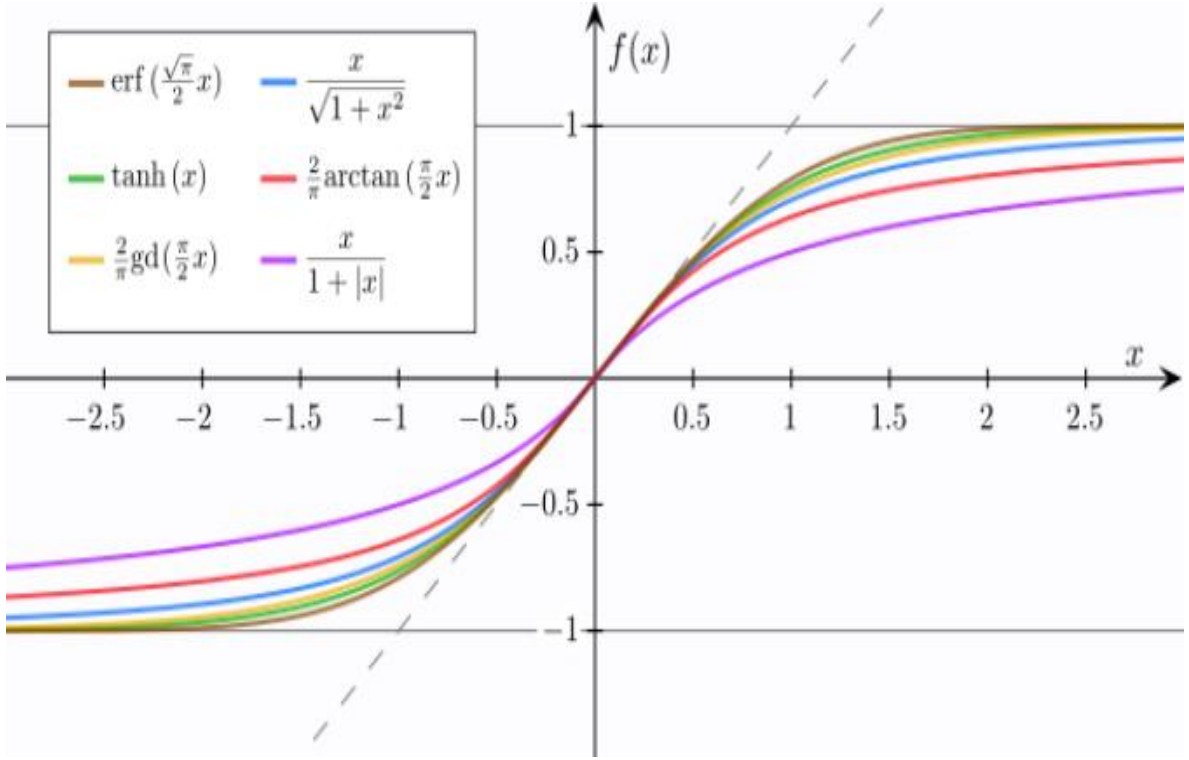
$$f(x) = \arctan x$$

- Gudermannian function

$$f(x) = \mathrm{gd}(x) = \int_0^x \frac{dt}{\cosh t}$$

- Error function

$$f(x) = \mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$$



Logistic函数与回归

- 对于一个二元逻辑回归，我们只需要确保

$$P(y = 1) + P(y = 0) = 1$$

$$P(y=1) = \sigma(z) = \frac{1}{1+\exp(-z)}$$

$$P(y=0) = 1 - \sigma(z) = 1 - \frac{1}{1+\exp(-z)} = \frac{\exp(-z)}{1+\exp(-z)}$$

- 根据 Sigmoid 函数的性质

$$1 - \sigma(x) = \sigma(-x)$$

风险函数

- 我们使用极大似然估计法来求解. 我们把单个样本看做一个事件, 那么这个事件发生的概率就是:

$$P(y|\boldsymbol{x}) = \begin{cases} p, y = 1 \\ 1 - p, y = 0 \end{cases}$$

  - 它等价于:

$$P(y_i|\boldsymbol{x}_i) = p^{y_i}(1-p)^{1-y_i}$$

  - 则组样本的概率:

$$\begin{aligned} P &= P(y_1|\boldsymbol{x}_1)P(y_2|\boldsymbol{x}_2)P(y_3|\boldsymbol{x}_3)\ldots P(y_m|\boldsymbol{x}_m) \\ &= \prod_{i=1}^{m} p^{y_i}(1-p)^{1-y_i} \end{aligned}$$

风险函数

- 等式两边同时取对数，就得到了分类问题最常用的风险函数：**交叉熵损失函数**：

$$\mathcal{R}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}\left(p_r(y^{(n)}=1|\boldsymbol{x}^{(n)})\log\hat{y}^{(n)} + p_r(y^{(n)}=0|\boldsymbol{x}^{(n)})\log(1-\hat{y}^{(n)})\right)$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}\log\hat{y}^{(n)} + (1-y^{(n)})\log(1-\hat{y}^{(n)})\right)$$

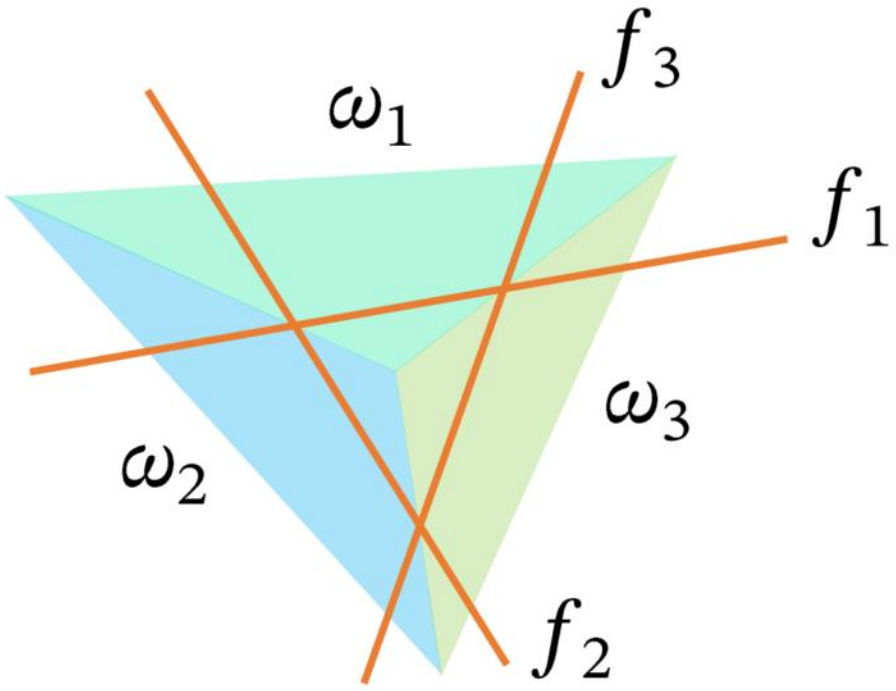
Softmax 回归

- Softmax函数

$$\mathrm{softmax}(x_k) = \frac{\exp(x_k)}{\sum_{i=1}^{K} \exp(x_i)}$$

- 给定一个样本x，Softmax回归预测的属于类别c的条件概率为

$$p(y = c|\boldsymbol{x}) = \mathrm{softmax}(\boldsymbol{w}_c^{\mathsf{T}}\boldsymbol{x})$$
$$= \frac{\exp(\boldsymbol{w}_c^{\mathsf{T}}\boldsymbol{x})}{\sum_{c'=1}^{C} \exp(\boldsymbol{w}_{c'}^{\mathsf{T}}\boldsymbol{x})},$$



参数学习

- 采用交叉熵损失函数，Softmax回归模型的风险函数为

$$\mathcal{R}(\boldsymbol{W}) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} \boldsymbol{y}_c^{(n)} \log \hat{\boldsymbol{y}}_c^{(n)} \qquad \hat{\boldsymbol{y}}^{(n)} = \mathrm{softmax}(\boldsymbol{W}^{\top}\boldsymbol{x}^{(n)})$$

$$= -\frac{1}{N}\sum_{n=1}^{N} (\boldsymbol{y}^{(n)})^{\top} \log \hat{\boldsymbol{y}}^{(n)},$$

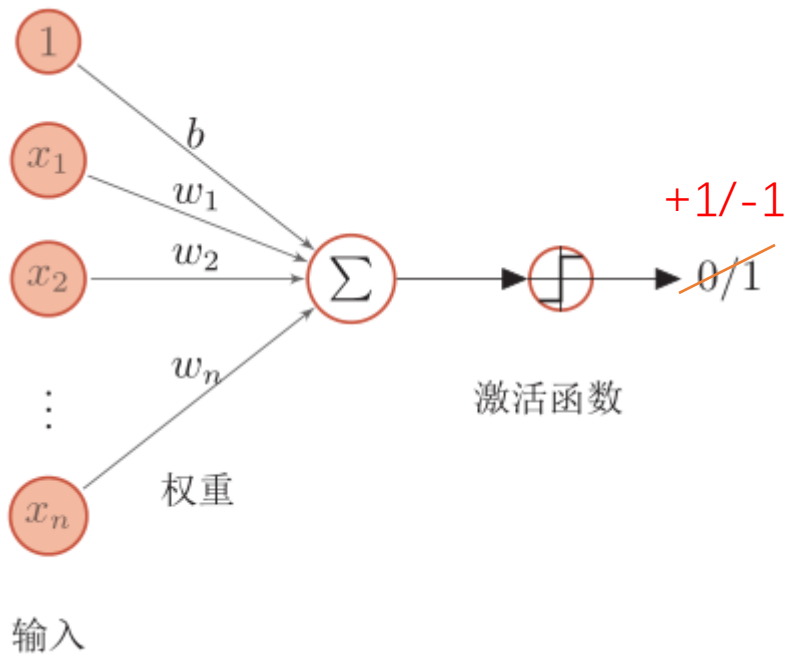
- 风险函数 $\mathcal{R}(W)$关于W的梯度为

$$\frac{\partial \mathcal{R}(\boldsymbol{W})}{\partial \boldsymbol{W}} = -\frac{1}{N}\sum_{n=1}^{N} \boldsymbol{x}^{(n)} \left(\boldsymbol{y}^{(n)} - \hat{\boldsymbol{y}}^{(n)}\right)^{\top}$$

感知器

- 由 Frank Roseblatt 于 1957 年提出， 是一种广泛使用的线性分类器．感知器可谓是最简单的人工神经网络，只有一个神经元
- 模拟生物神经元行为的机器，有与生物神经元相对应的部件，如权重（突触）、偏置（阈值）及激活函

$$\hat{y} = \mathrm{sgn}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}) = \begin{cases} +1 & \text{当} \mathbf{w}^{\mathrm{T}}\mathbf{x} > 0 \\ -1 & \text{当} \mathbf{w}^{\mathrm{T}}\mathbf{x} \leq 0 \end{cases},$$



感知器

- 学习算法
  - 感知器的学习算法是一种错误驱动的在线学习算法：
  - 先初始化一个权重向量$\boldsymbol{w} \leftarrow 0$（通常是全零向量）；
  - 每次分错一个样本$(\boldsymbol{x}, y)$时，即 $y\boldsymbol{w}^T\boldsymbol{x} < \mathbf{0}$

  - 用这个样本来更新权重

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + y\boldsymbol{x}$$

  - 根据感知器的学习策略，可以反推出感知器的损失函数为

$$\mathcal{L}(\boldsymbol{w}; \boldsymbol{x}, y) = \max(0, -y\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})$$

支持向量机(SVM)

- 是在分类与回归分析中分析数据的监督式学习模型与相关的学习算法。
- 支持向量机的分类方法，是在一组分布中找出一个超平面作为决策边界，使模型在数据上的分类误差尽量接近于零
- 支持向量机在线性和非线性分类中，效果都非常好。

支持向量机

- 给定一个二分类器数据集

$$\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N} \qquad y_n \in \{+1, -1\}$$

- 如果两类样本是线性可分的，即存在一个超平面

$$\boldsymbol{w}^{\top}\boldsymbol{x} + b = 0$$

- 将两类样本分开，那么对于每个样本都有

$$y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b) > 0$$

- 每个样本 $x^{(n)}$ 到分割超平面的距离为

$$\gamma^{(n)} = \frac{|\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b|}{\|\boldsymbol{w}\|} = \frac{y^{(n)}(\boldsymbol{w}^{\top}\boldsymbol{x}^{(n)} + b)}{\|\boldsymbol{w}\|}$$

支持向量机

- 支持向量机的目标是寻找一个超平面$(\boldsymbol{w}^*, b^*)$使得γ最大，即

$$\max_{\boldsymbol{w},b} \quad \gamma$$
$$\text{s.t.} \quad \frac{y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b)}{\|\boldsymbol{w}\|} \geq \gamma, \forall n \in \{1, \cdots, N\}.$$

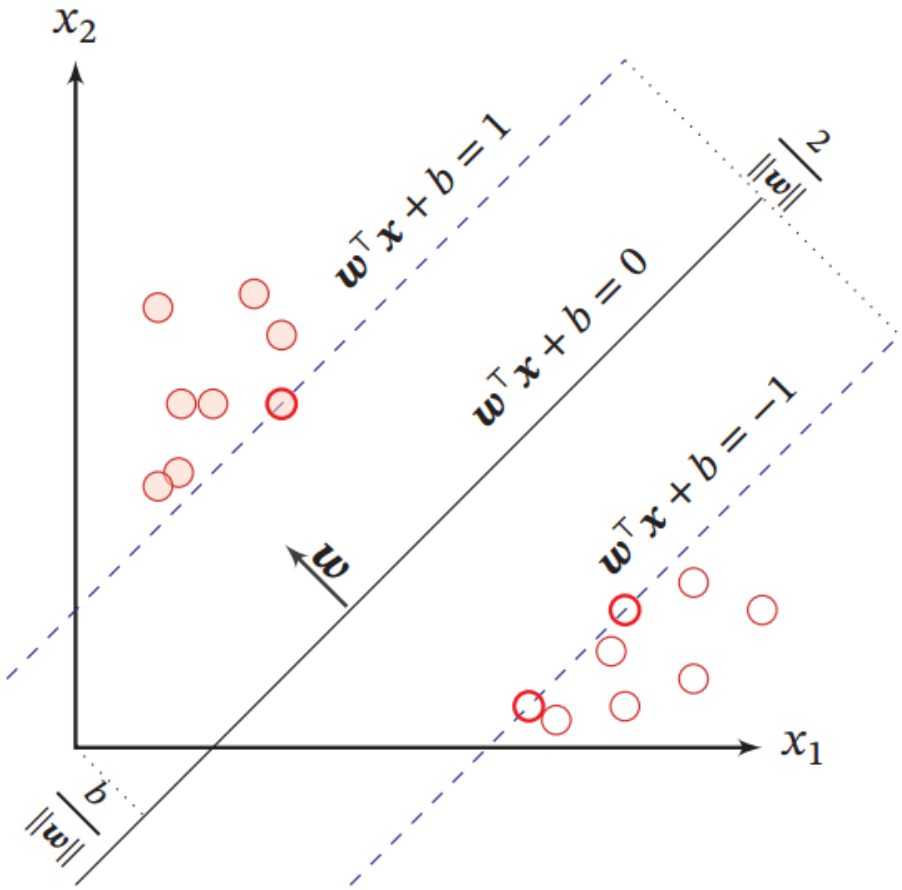
- 等价于

$$\max_{\boldsymbol{w},b} \quad \frac{1}{\|\boldsymbol{w}\|^2}$$
$$\text{s.t.} \quad y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b) \geq 1, \forall n \in \{1, \cdots, N\}.$$

- 数据集中所有满足$y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b) = 1$的样本点，都称为**支持向量**

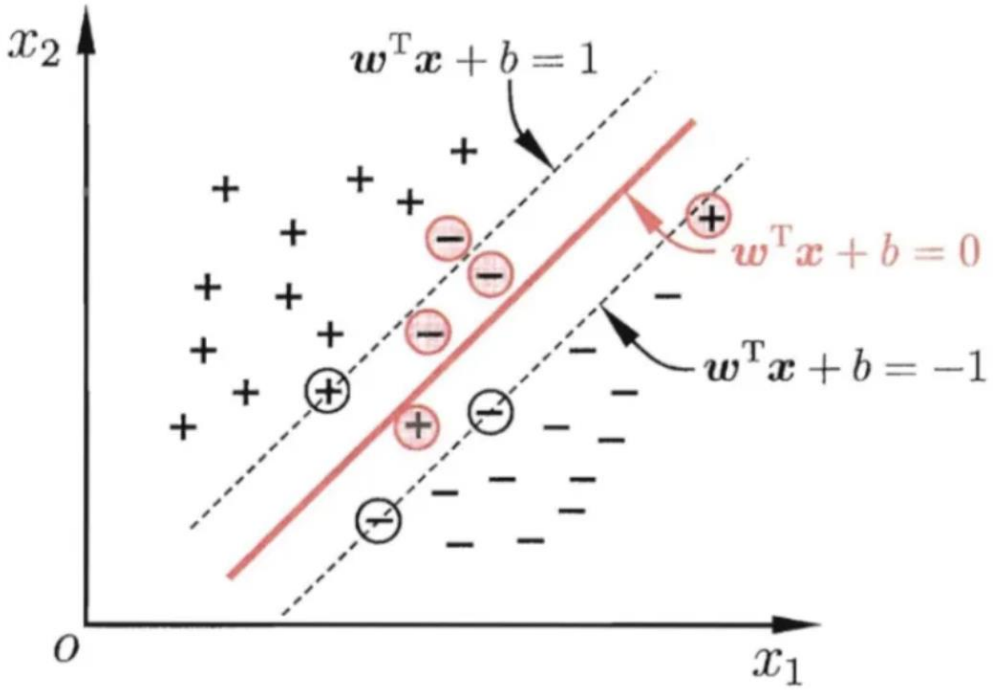
支持向量机

- 其分割超平面有很多个，但是间隔最大的超平面是唯一的。



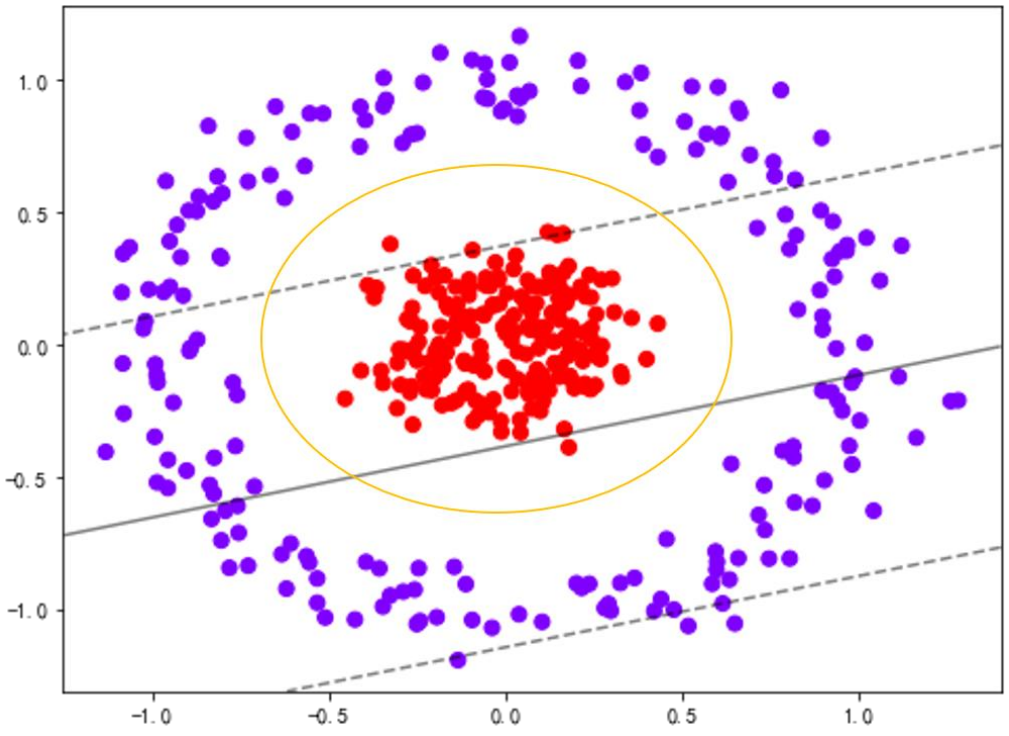
支持向量机

- 在最大化间隔的同时，不满足约束的样本应尽可能地少。



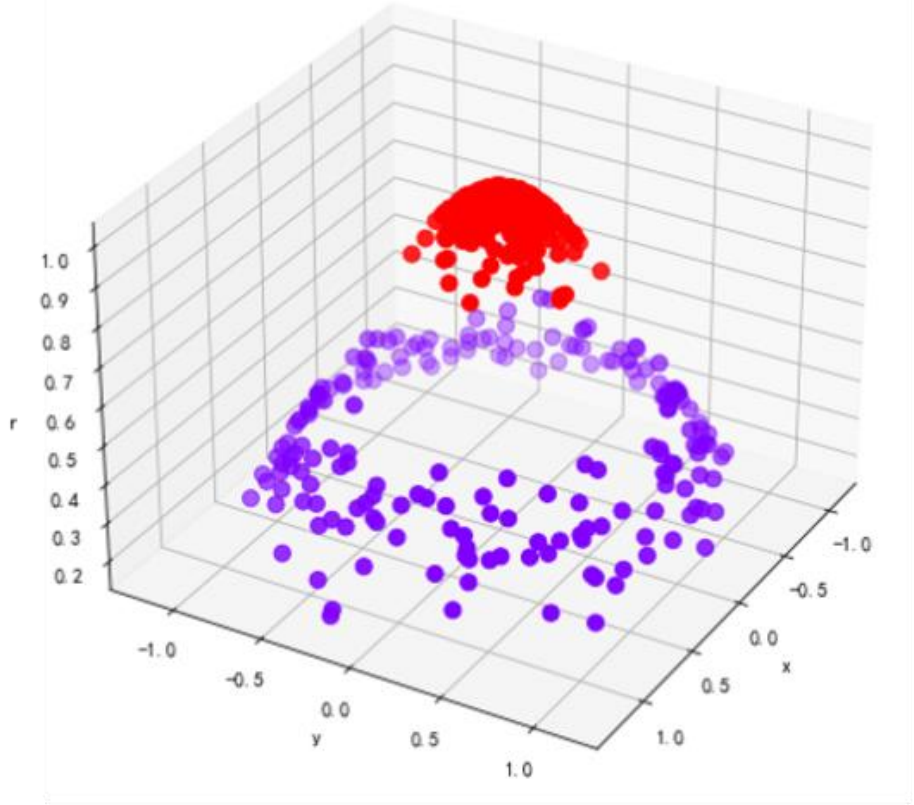
非线性支持向量机

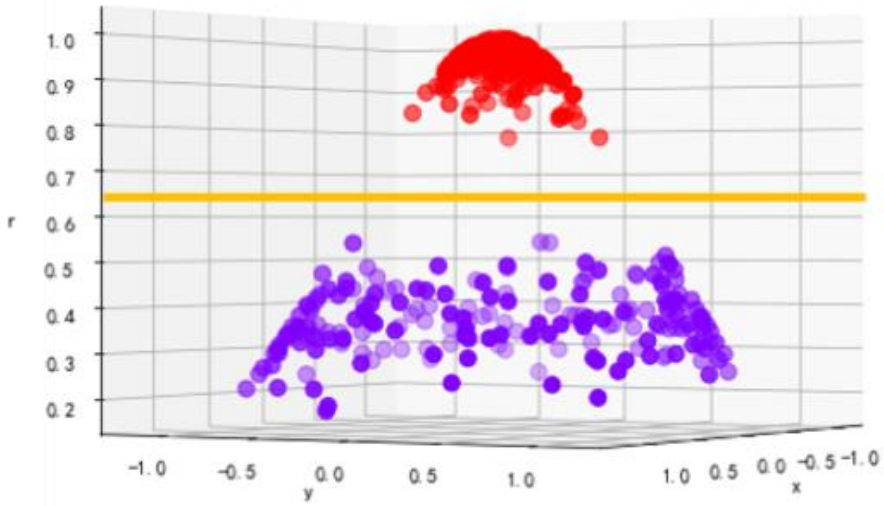
- 如果一组数据无法找到一个超平面进行分离，那么如何利用支持向量机模型处理这类分类问题？



非线性支持向量机

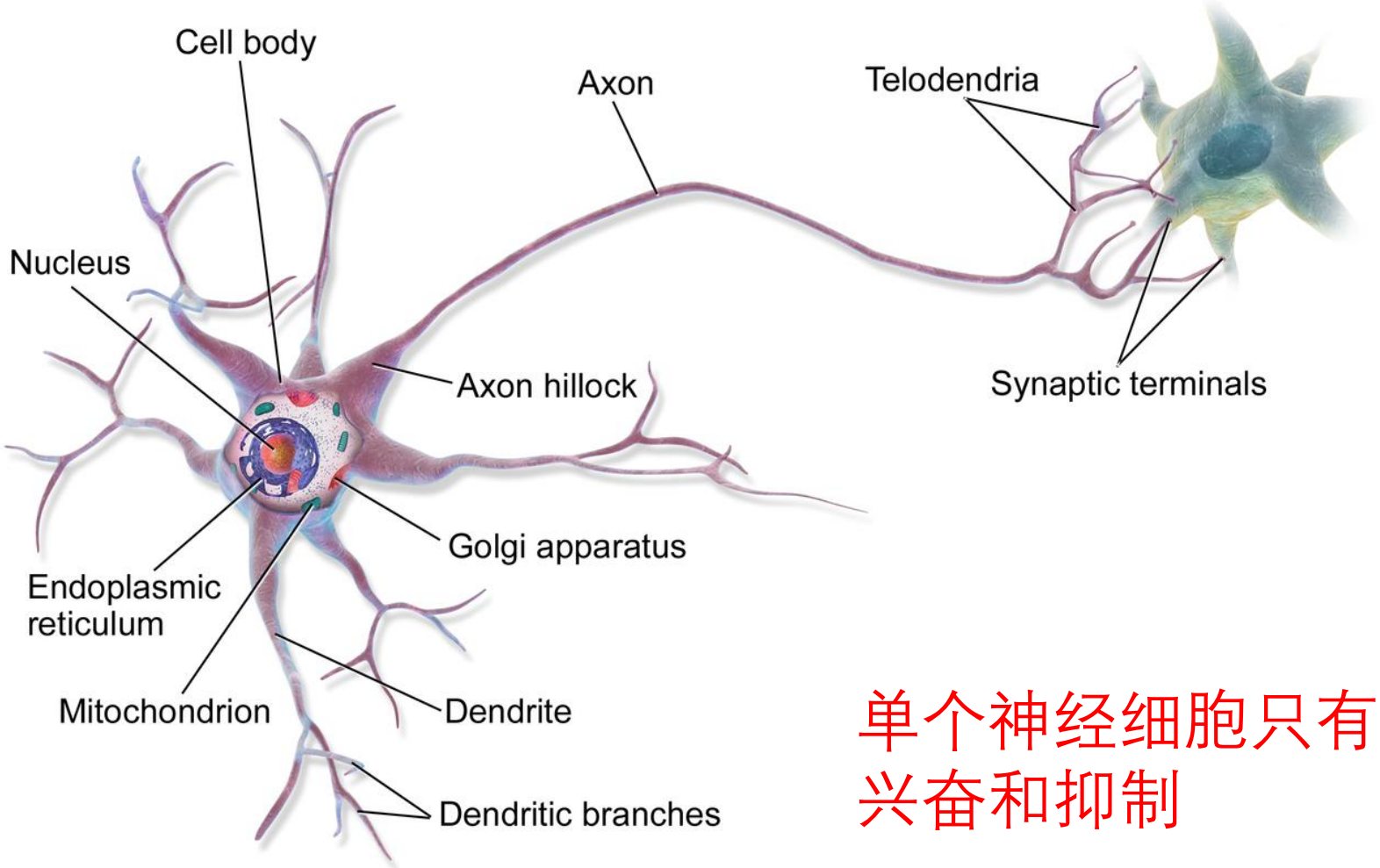
- 原理利用核技巧，将数据从原始的空间投射到新空间中
- 这种非线性变换，将非线性问题变换成线性问题





神经网络

生物神经元



单个神经细胞只有两种状态:
兴奋和抑制

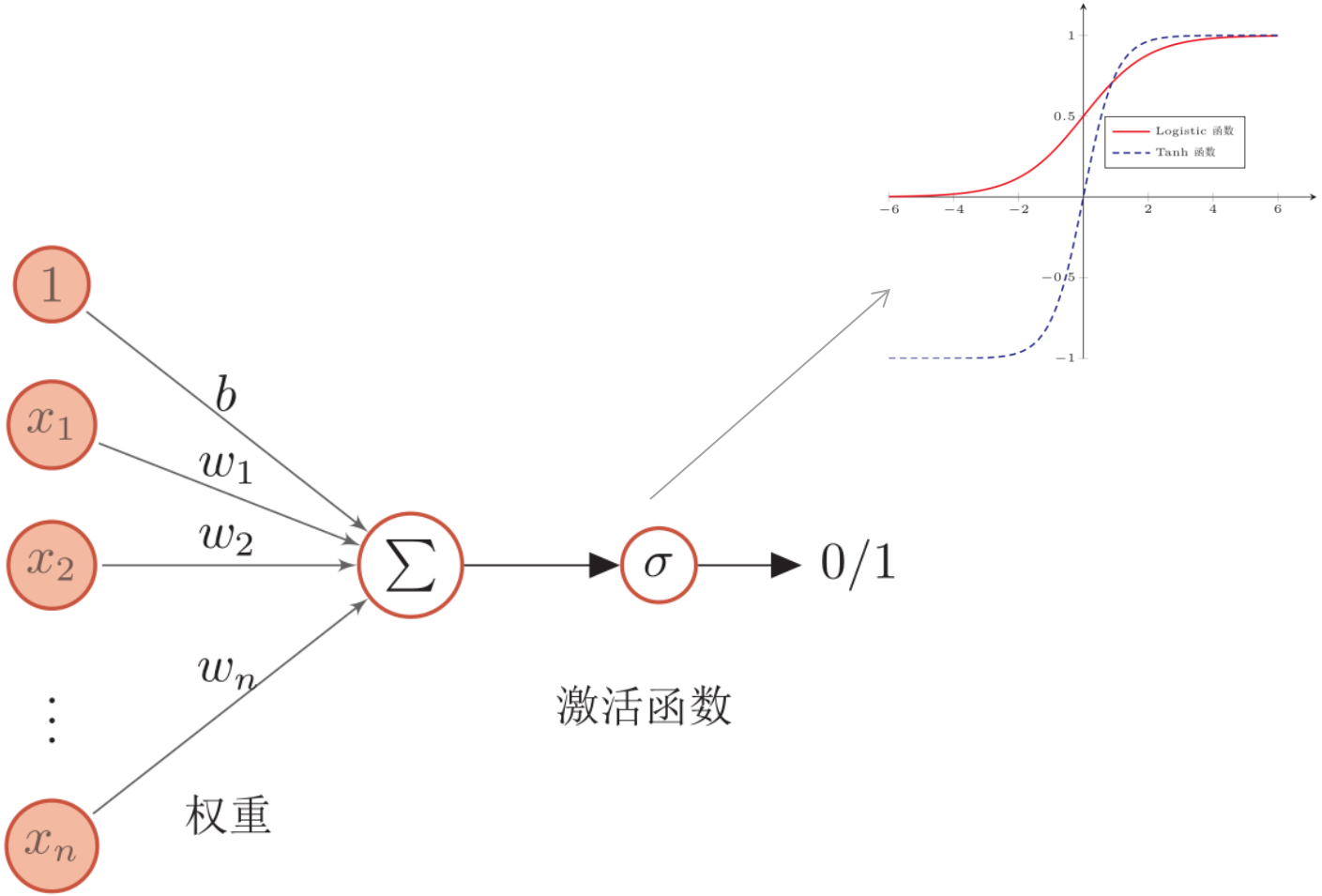
神经网络如何学习？

- 赫布法则 Hebb's Rule
  - “当神经元 A的一个轴突和神经元B很近，足以对它产生影响，并且持续地、重复地参与了对神经元B的兴奋，那么在这两个神经元或其中之一会发生某种生长过程或新陈代谢变化，以致于神经元A作为能使神经元B兴奋的细胞之一，它的效能加强了。”

----加拿大心理学家Donald Hebb，
《行为的组织》，1949

- 人脑有两种记忆：长期记忆和短期记忆。短期记忆持续时间不超过一分钟。如果一个经验重复足够的次数，此经验就可储存在长期记忆中。
- 短期记忆转化为长期记忆的过程就称为凝固作用。
- 人脑中的海马区为大脑结构凝固作用的核心区域。

人工神经元



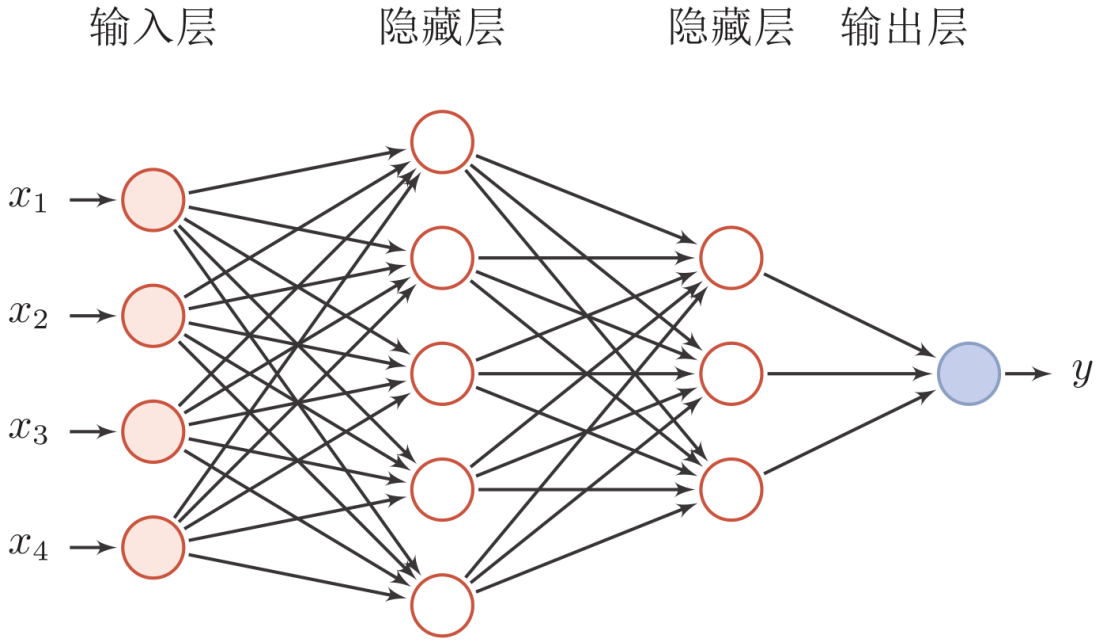
人工神经网络

- 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。这种由许多神经元组成的信息处理网络具有并行分布结构，因此可以进行并行计算。
- 设计人工神经网络主要考虑三方面：
  - **神经元的激活规则：**主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
  - **网络的拓扑结构：**不同神经元之间的连接关系。
  - **学习方法：**选择合适的训练数据以及优化算法来训练神经网络的参数。

前馈神经网络

- 各神经元分别属于不同的层，层内无连接。
- 相邻两层之间的神经元全部两两连接。
- 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。

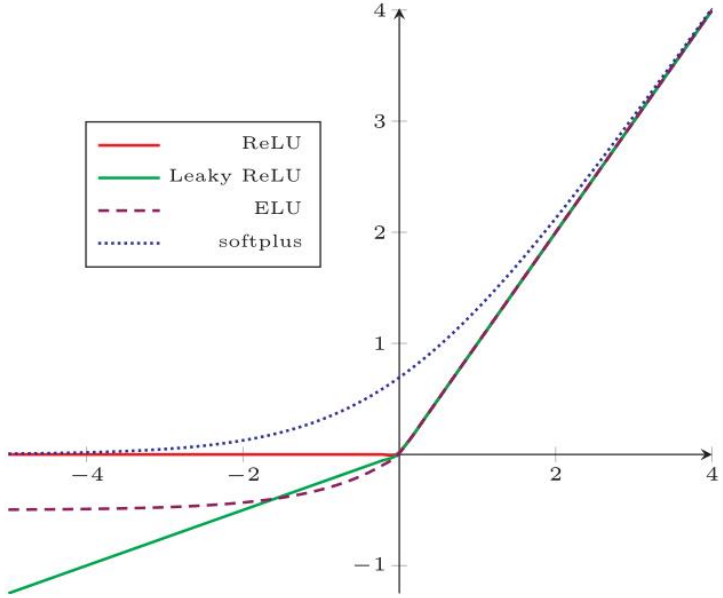
$$y = (f^3(f^2(f^1(x))))$$



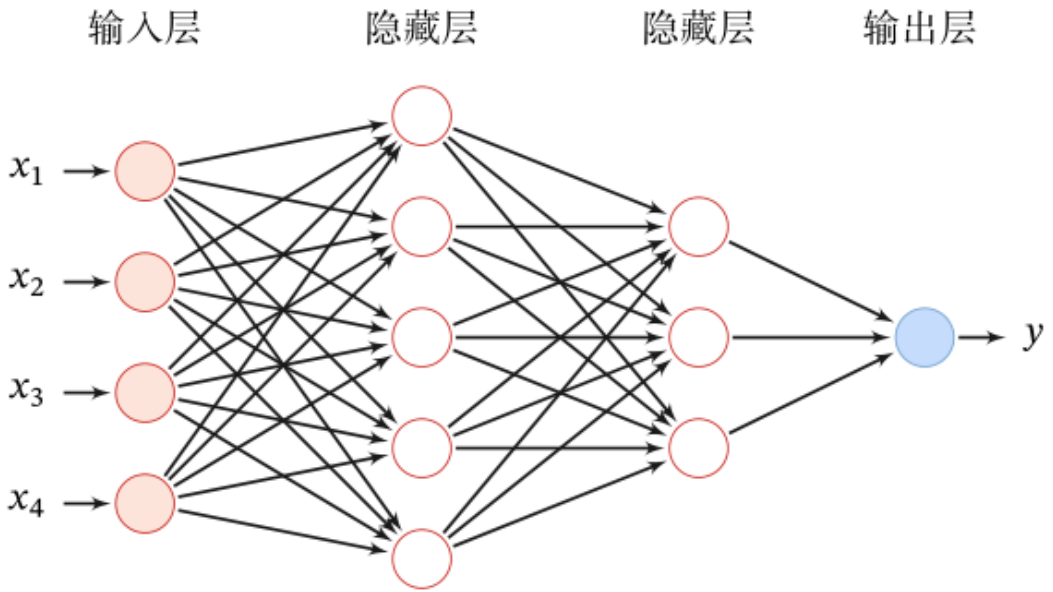
激活函数

- 是神经网络中的一个关键组件。它是一种非线性函数，用于对神经网络中神经元的输入进行变换，从而产生神经元的输出。
- 生物学合理性
  - 单侧抑制、宽兴奋边界
- 最常见的激活函数：

$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$



前馈神经网络



给定一个前馈神经网络，用下面的记号来描述这样网络：

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\boldsymbol{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\boldsymbol{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\boldsymbol{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

通用近似定理

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989, Hornik et al., 1989]： 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0,1]^d$，$C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$，存在一个整数 m，和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$，$i = 1, \cdots, m$，以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^{m} v_i \varphi(\mathbf{w}_i^{\mathrm{T}} \mathbf{x} + b_i), \tag{4.33}$$

作为函数 f 的近似实现，即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \tag{4.34}$$

其中 $\epsilon > 0$ 是一个很小的正数。

应用到机器学习

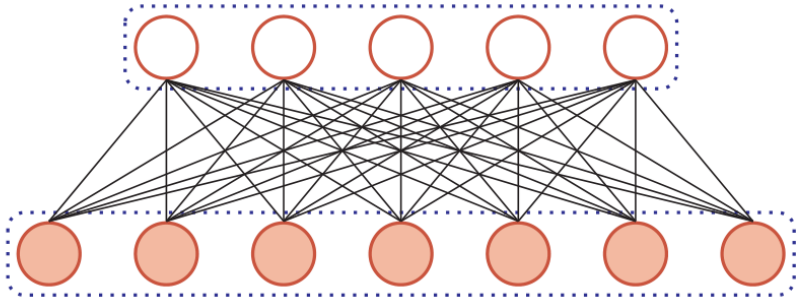
- 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\varphi(\mathbf{x}), \theta)$$

分类器　　神经网络

全连接前馈神经网络

- 权重矩阵的参数非常多



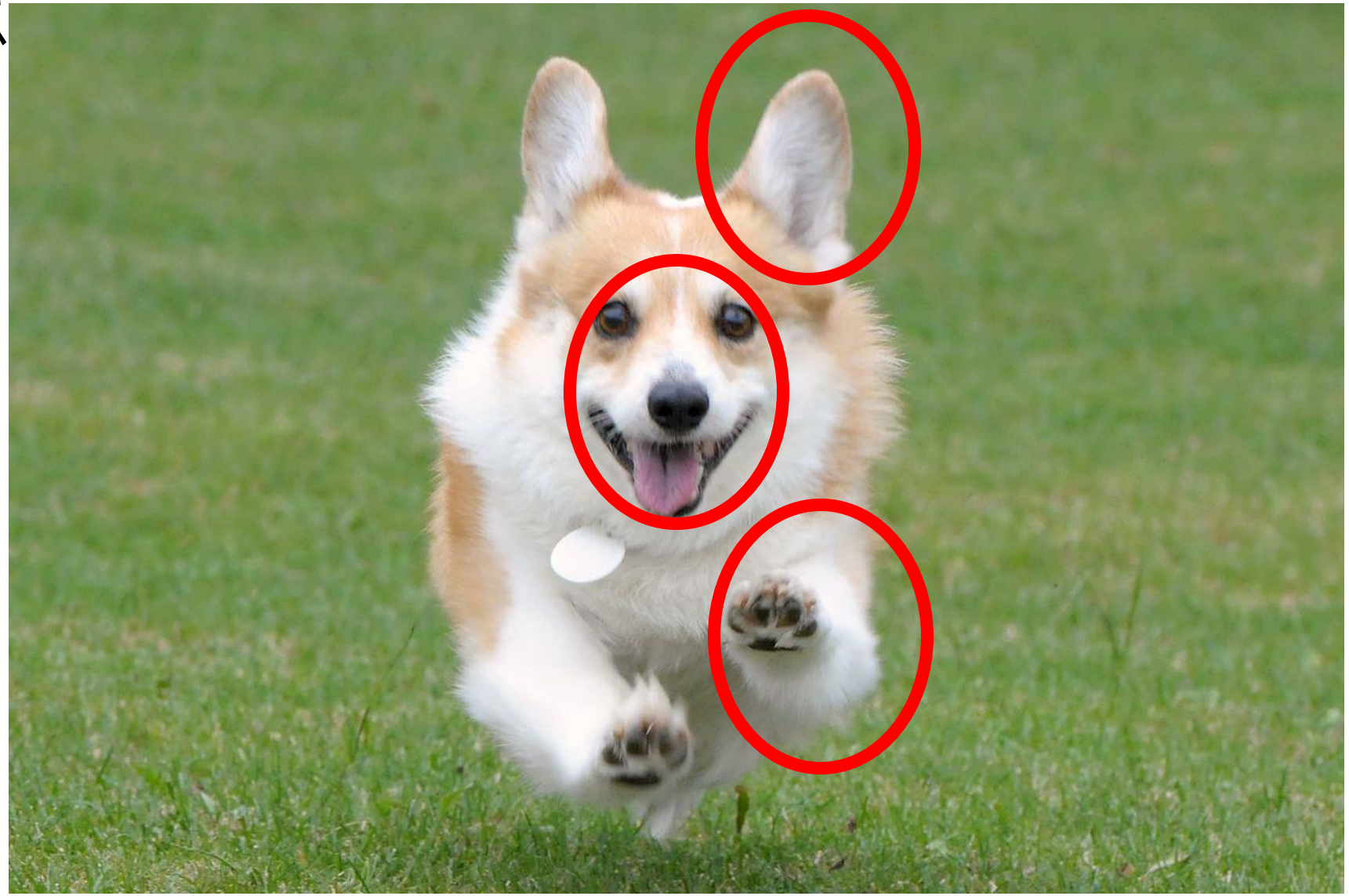
- 局部不变性特征
  - 自然图像中的物体都具有局部不变性特征
    - 尺度缩放、平移、旋转等操作不影响其语义信息。
  - 全连接前馈网络很难提取这些局部不变特征

卷积神经网络

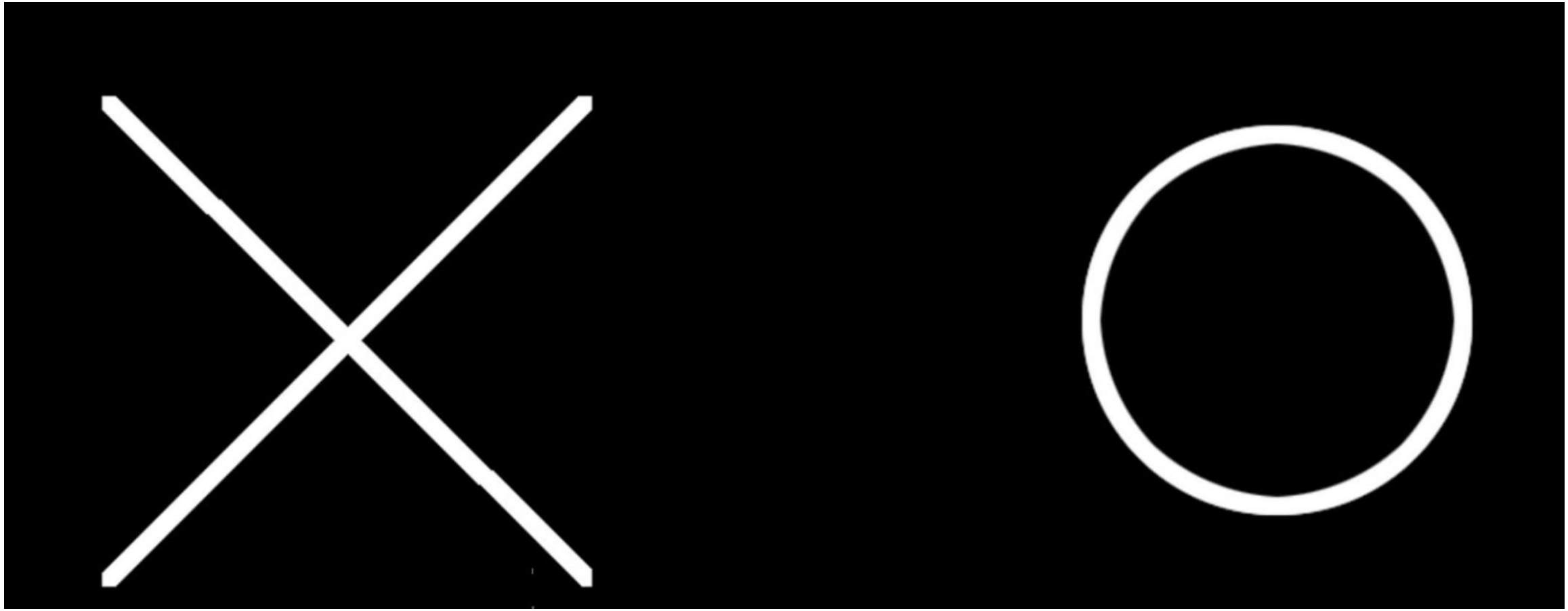
- 卷积神经网络（Convolutional Neural Networks，CNN）
  - 一种前馈神经网络
  - 受生物学上感受野（Receptive Field）的机制而提出的
    - 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。
  - 特别适用于处理具有网格拓扑结构的数据，如图像（2D网格）和时间序列数据（1D网格）
- 卷积神经网络有三个结构上的特性：
  - 局部连接
  - 权重共享
  - 空间或时间上的次采样

卷积

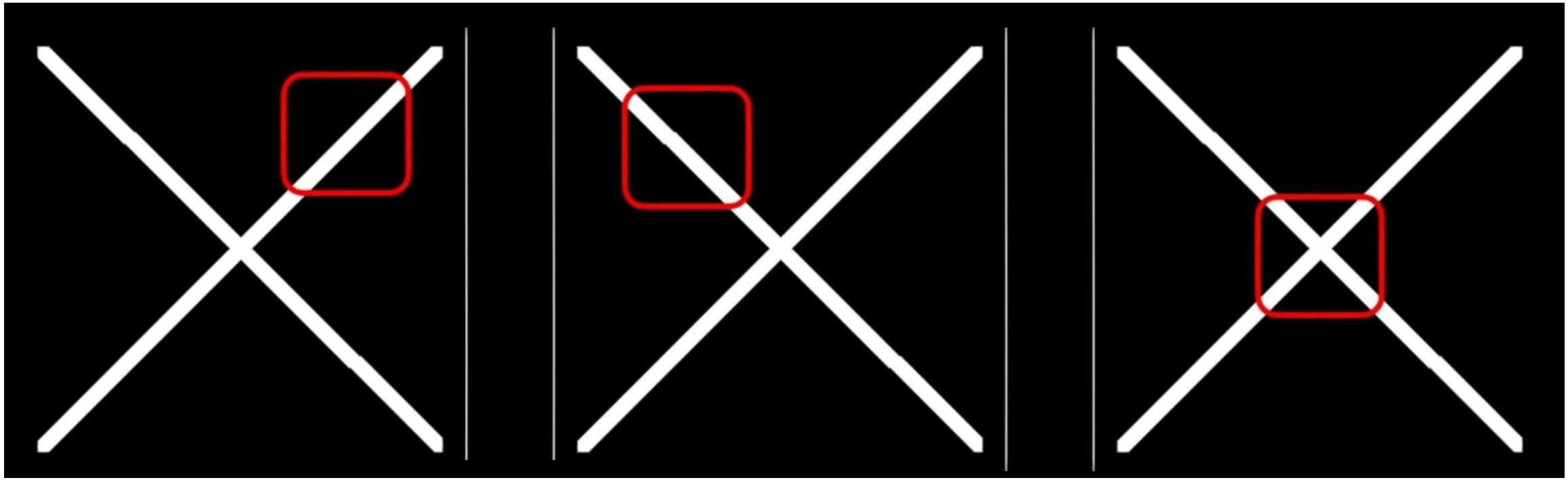
卷积



卷积



卷积



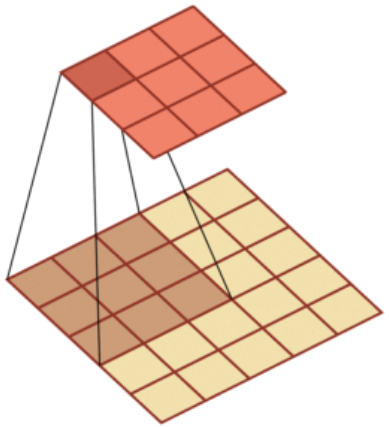
二维卷积

- 在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此我们需要二维卷积。

一个输入信息 $\boldsymbol{X}$ 和滤波器 $\boldsymbol{W}$ 的二维卷积定义为

$$\boldsymbol{Y} = \boldsymbol{W} * \boldsymbol{X},$$

$$y_{ij} = \sum_{u=1}^{U} \sum_{v=1}^{V} w_{uv} x_{i-u+1,j-v+1}.$$

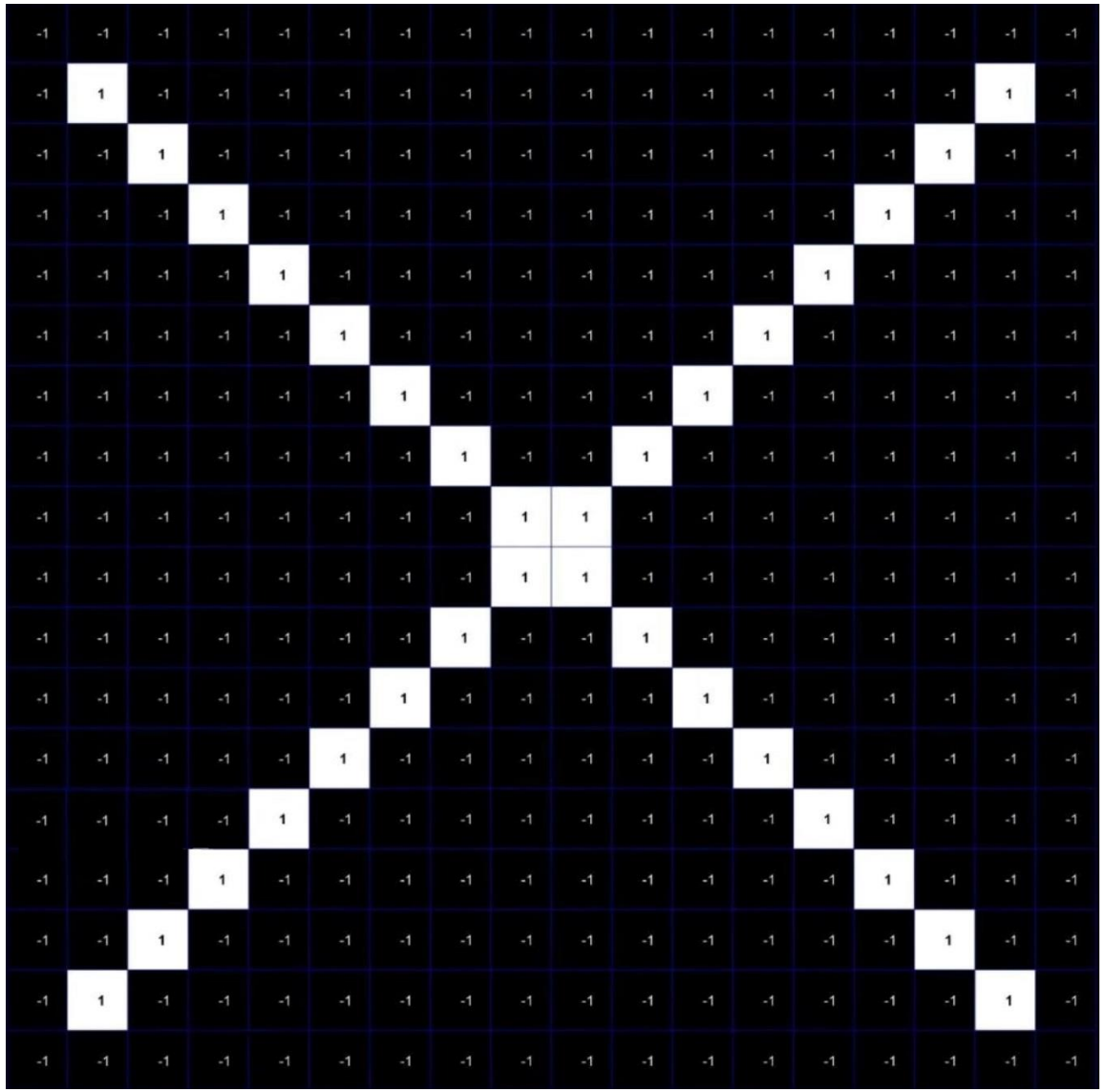


$$\begin{bmatrix} 1 & 1 & \underset{\times -1}{1} & \underset{\times 0}{1} & \underset{\times 0}{1} \\ -1 & 0 & \underset{\times 0}{-3} & \underset{\times 0}{0} & \underset{\times 0}{1} \\ 2 & 1 & \underset{\times 0}{1} & \underset{\times 0}{-1} & \underset{\times 1}{0} \\ 0 & -1 & 1 & 2 & 1 \\ 1 & 2 & 1 & 1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & -2 & -1 \\ 2 & 2 & 4 \\ -1 & 0 & 0 \end{bmatrix}$$

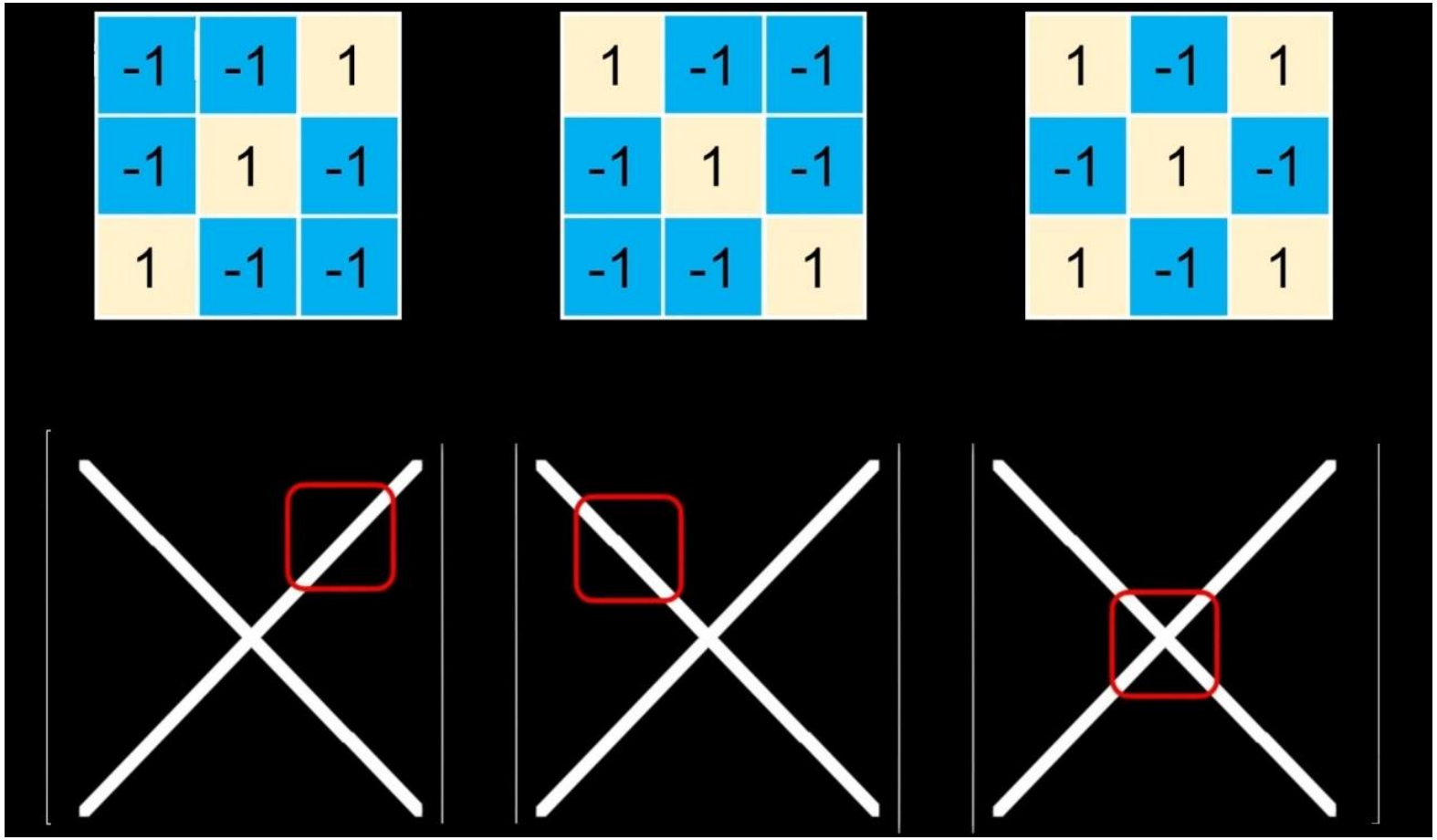
卷积类型

- 卷积的结果按输出长度不同可以分为三类：
  - 窄卷积：步长 $T = 1$，两端不补零 $P = 0$，卷积后输出长度为 $M - K + 1$
  - 宽卷积：步长 $T = 1$，两端补零 $P = K - 1$，卷积后输出长度 $M + K - 1$
  - 等宽卷积：步长 $T = 1$，两端补零 $P = (K - 1)/2$，卷积后输出长度 M

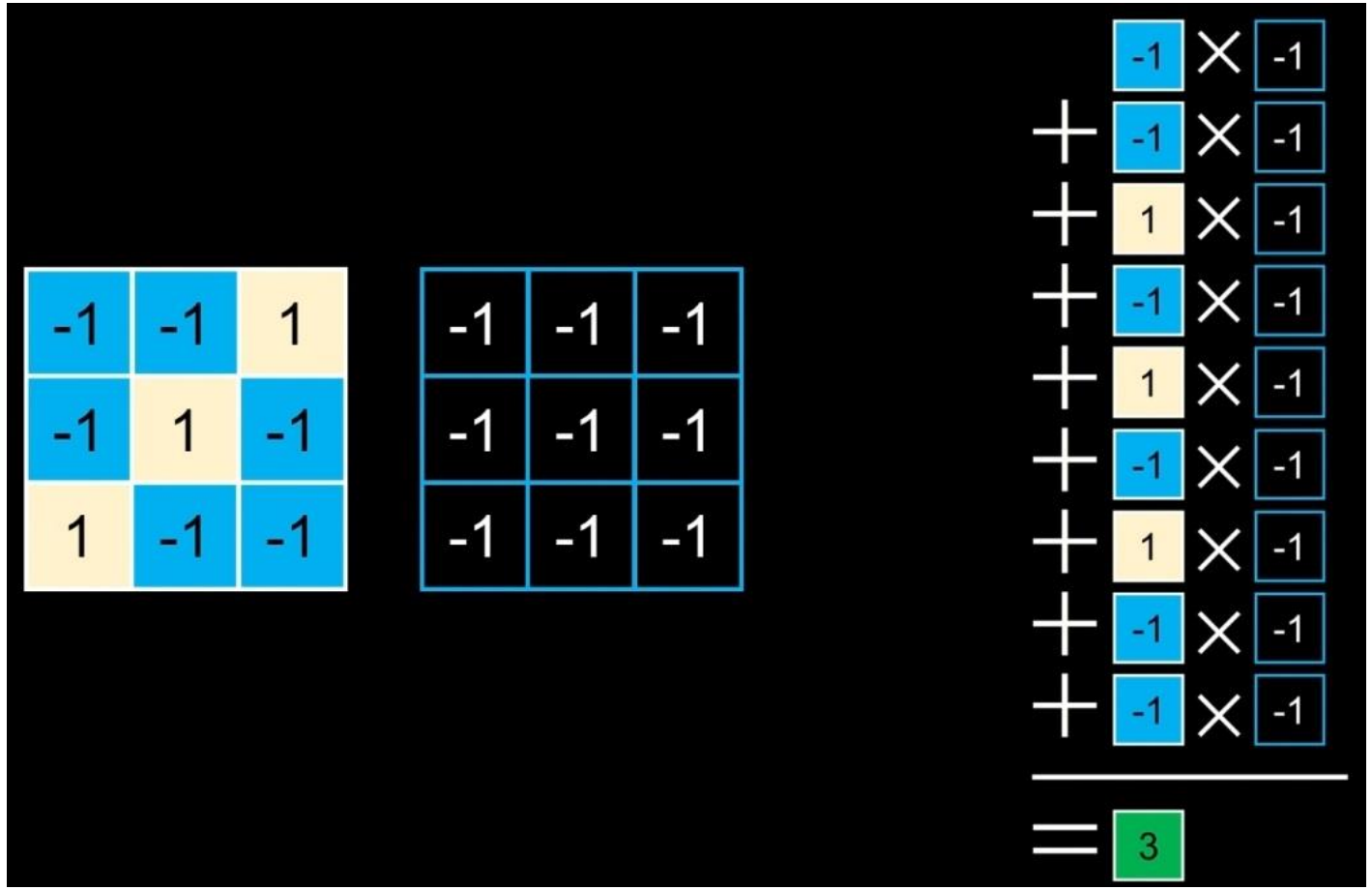
卷积


-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1
-1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1
-1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1
-1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1
-1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 1 1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 -1 1 1 -1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 -1 1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1
-1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1
-1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1
-1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1
-1 -1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1
-1 1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1

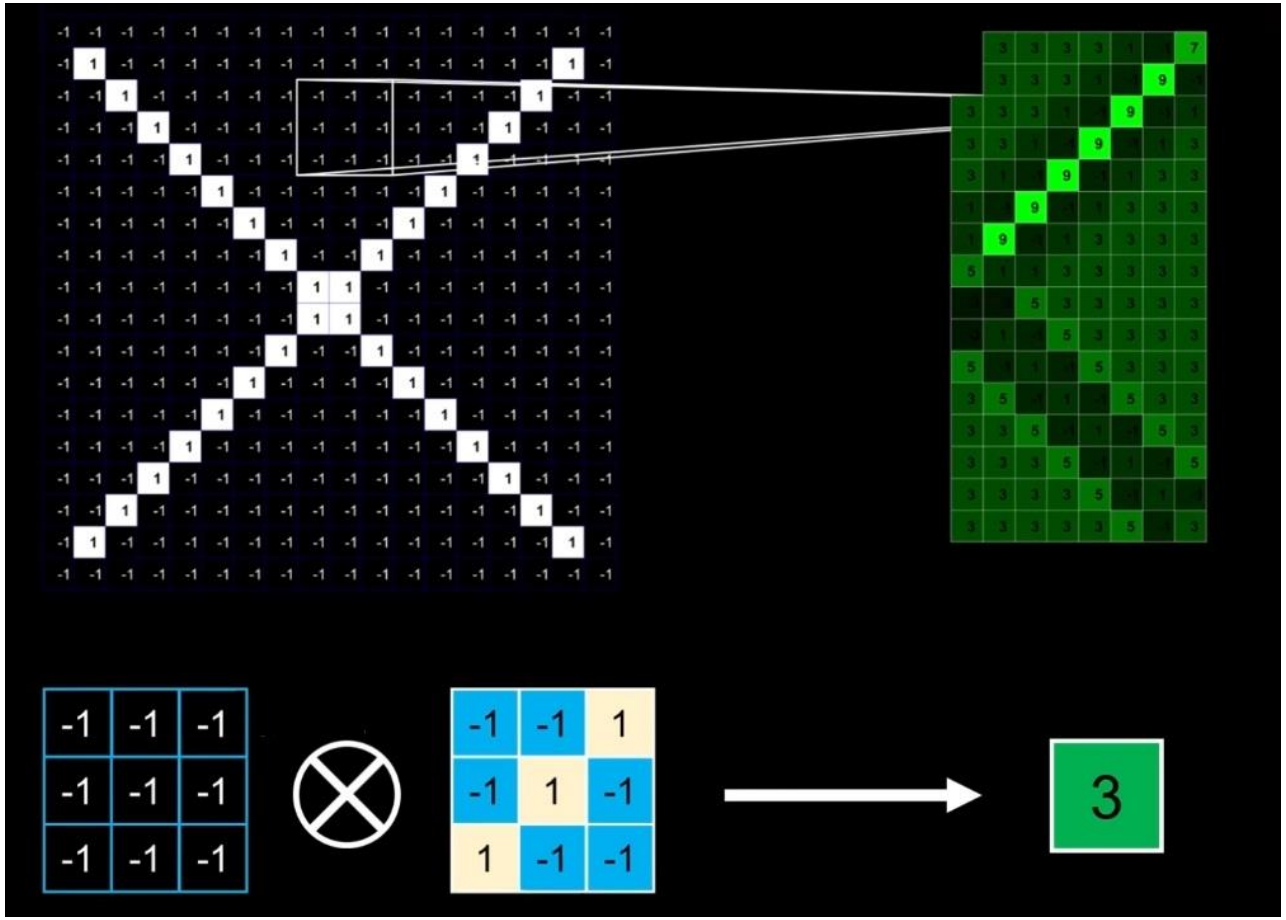
卷积核作为特征提取器



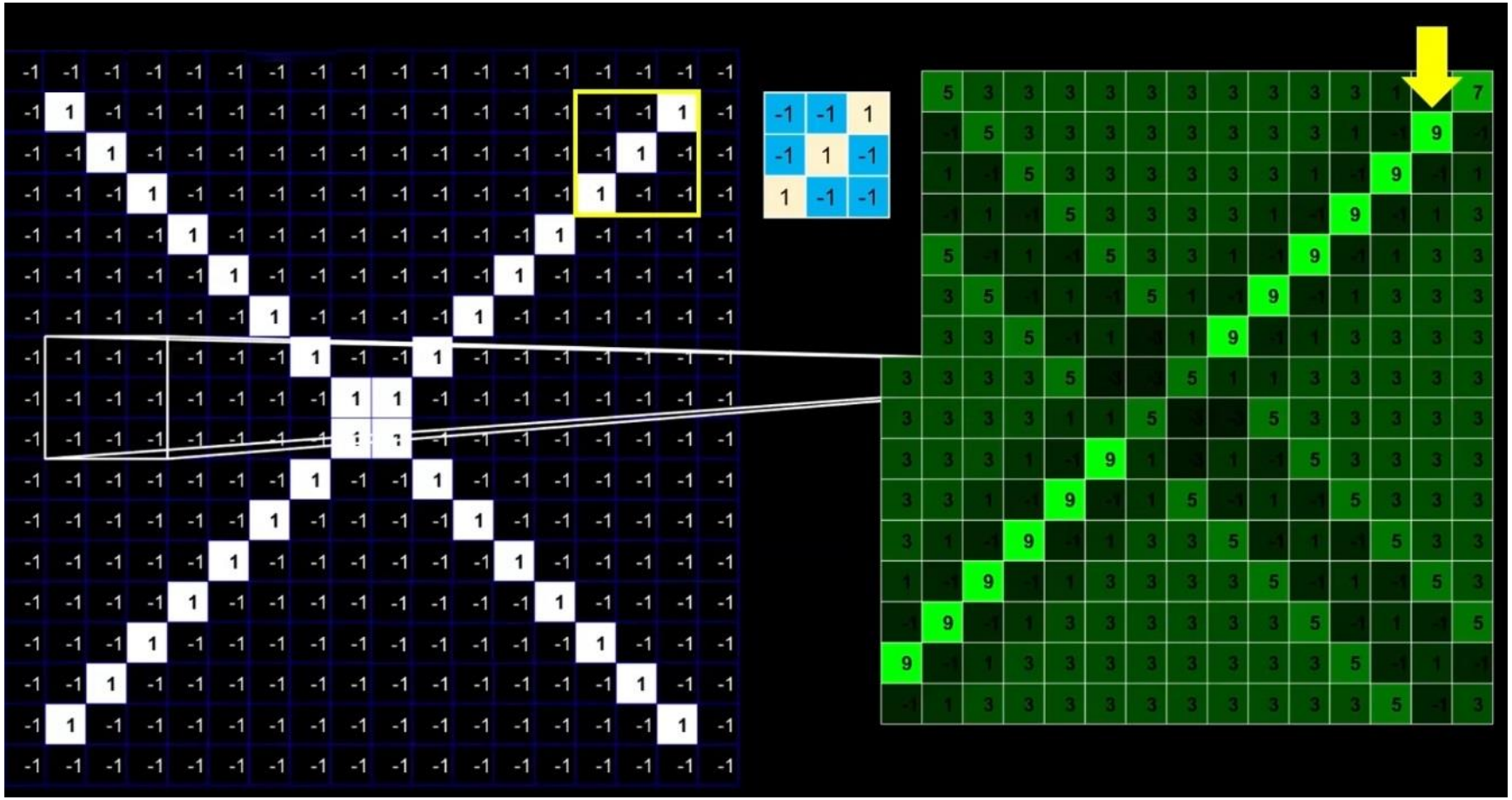
二维卷积



二维卷积

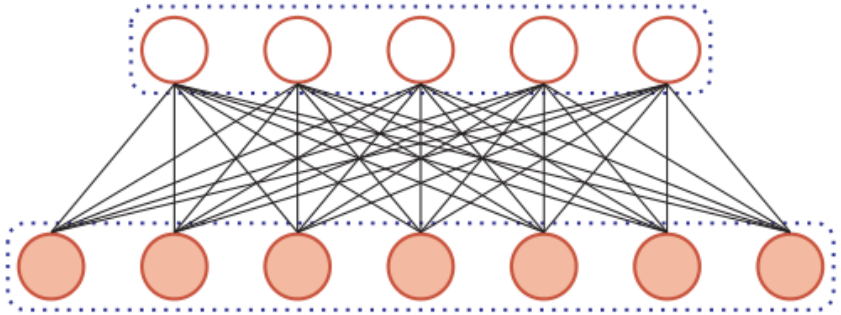


卷积核作为特征提取器

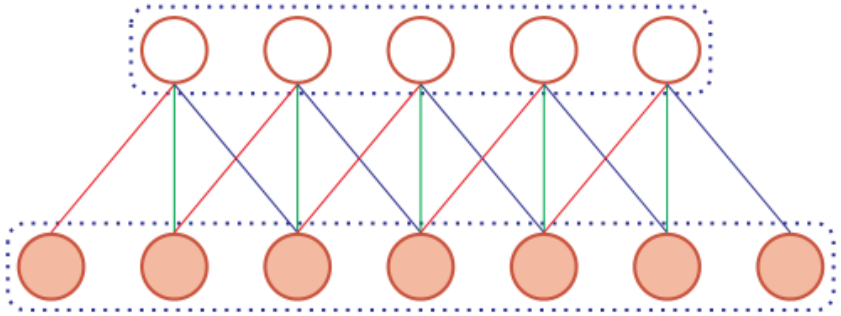


卷积神经网络

- 用卷积层代替全连接层



(a) 全连接层



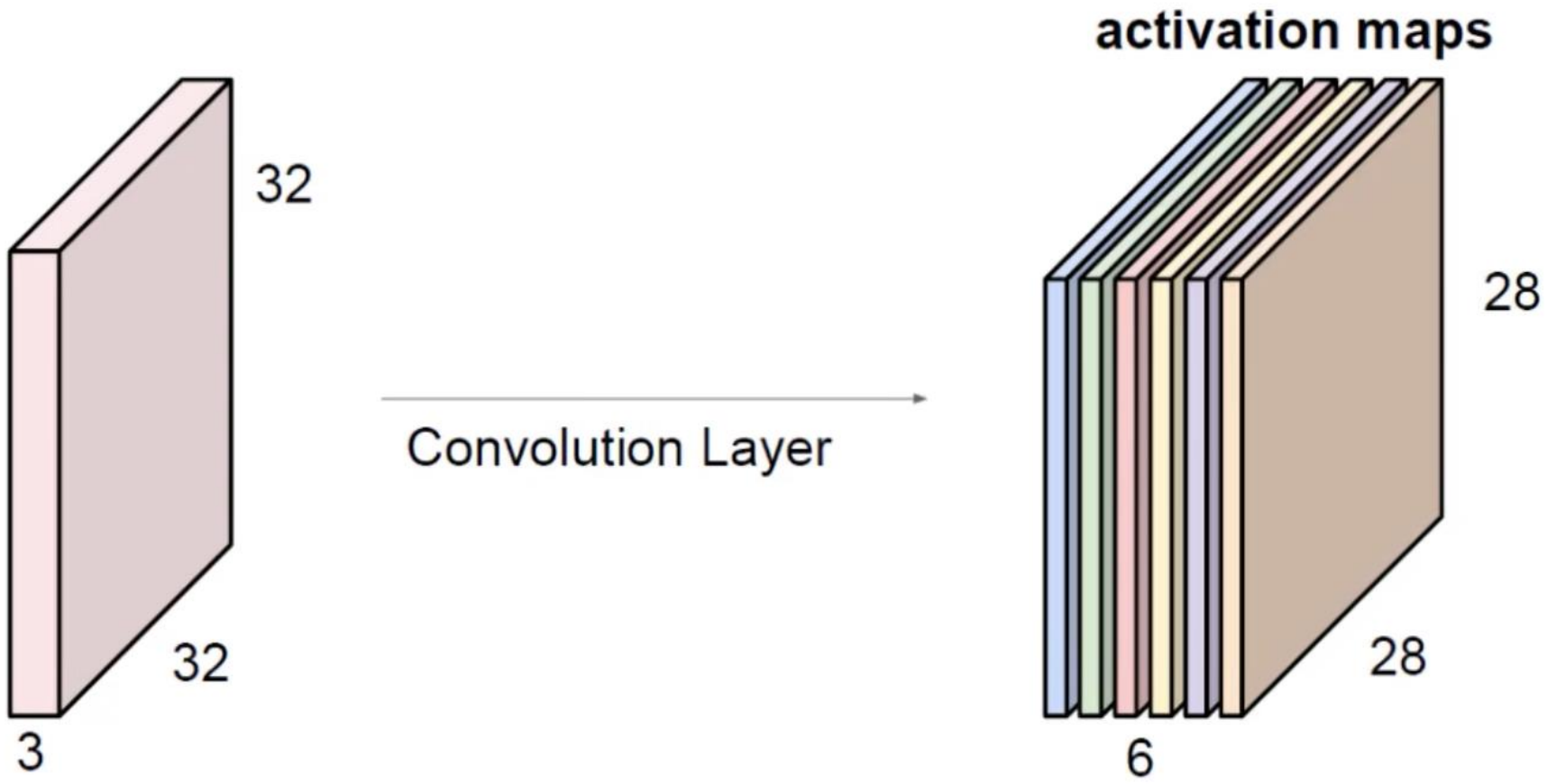
(b) 卷积层

多个卷积核

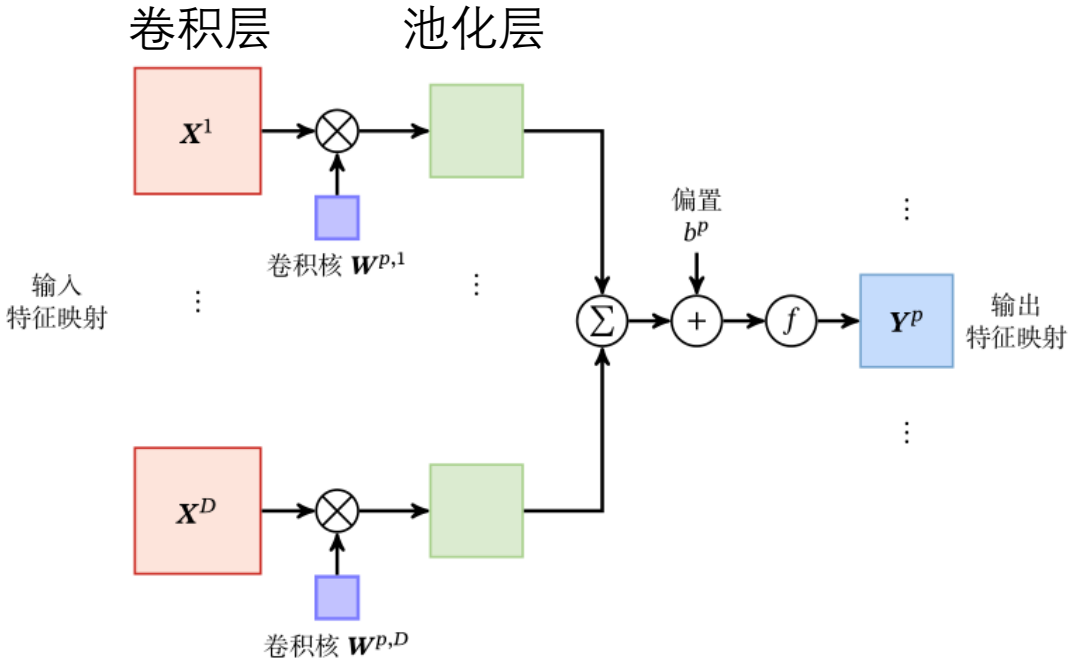
- 特征映射（Feature Map）： 图像经过卷积后得到的特征。
  - 卷积核看成一个特征提取器

- 卷积层
  - 输入：D个特征映射 $M \times N \times D$
  - 输出：P个特征映射 $M' \times N' \times P$

多个卷积核

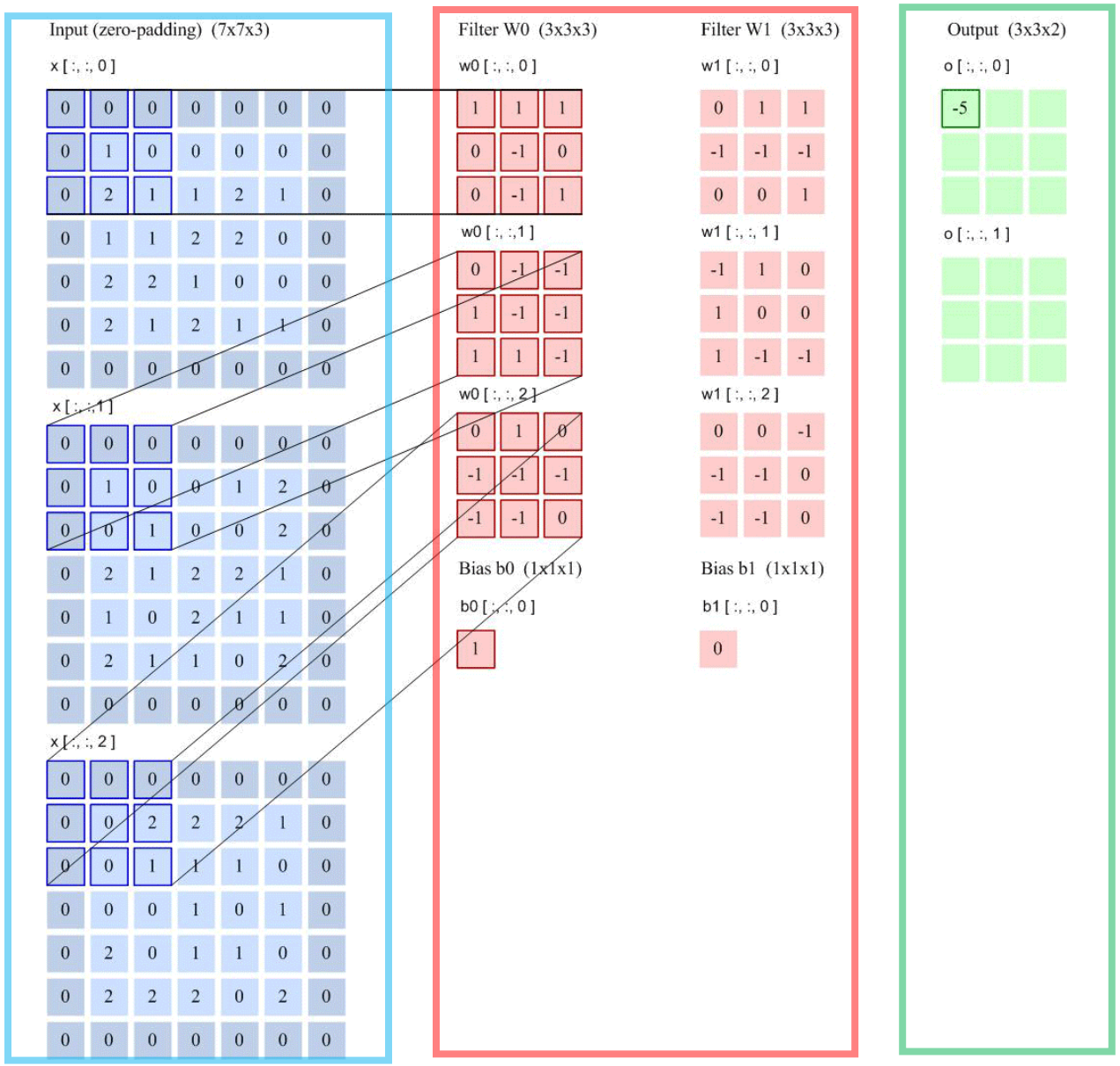


卷积层的映射关系



$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^{D} W^{p,d} \otimes X^d + b^p,$$

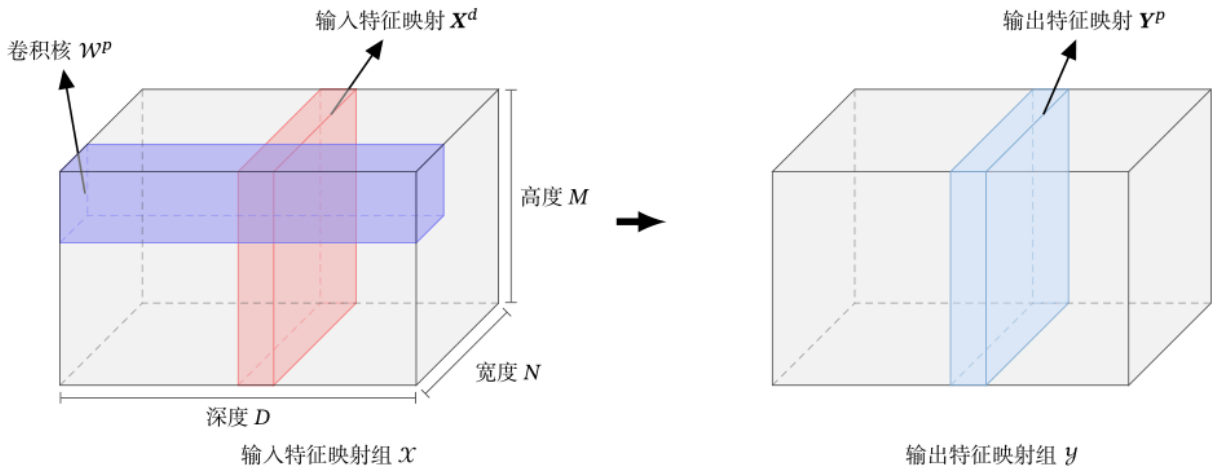
$$Y^p = f(Z^p).$$



步长2
filter 3*3
filter个数6
零填充 1

卷积层

- 典型的卷积层为3维结构

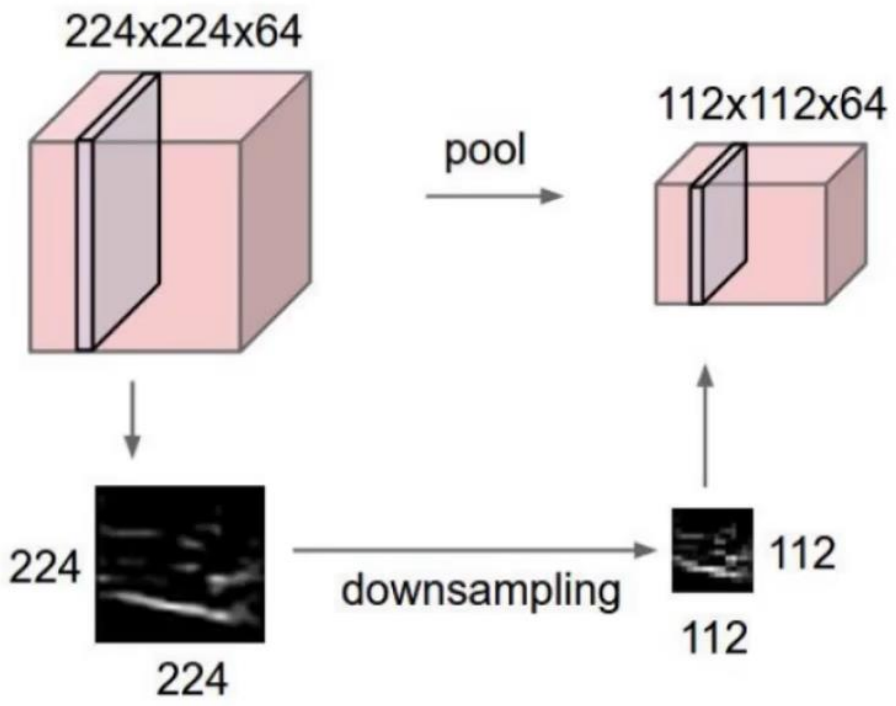


$$\boldsymbol{Z}^p = \boldsymbol{W}^p \otimes \boldsymbol{X} + b^p = \sum_{d=1}^{D} \boldsymbol{W}^{p,d} \otimes \boldsymbol{X}^d + b^p,$$

$$\boldsymbol{Y}^p = f(\boldsymbol{Z}^p).$$

池化层/汇聚层

- 卷积层虽然可以显著减少连接的个数，但是每一个特征映射的神经元个数并没有显著减少。

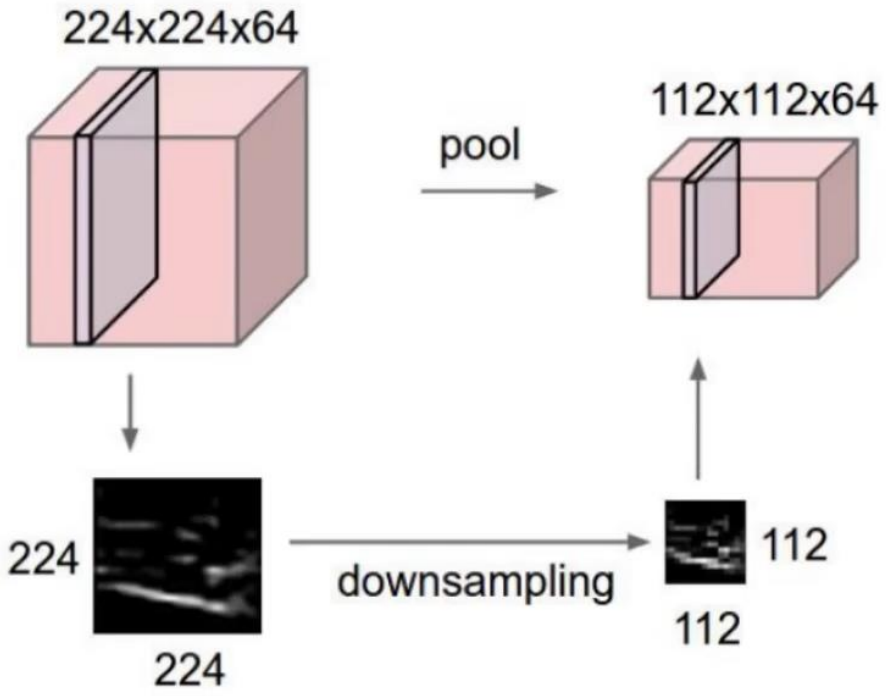


1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

7	

池化层/汇聚层

- 卷积层虽然可以显著减少连接的个数，但是每一个特征映射的神经元个数并没有显著减少。



1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

7	9
8	4

卷积神经网络

- 卷积层（Convolutional Layer）：卷积层通过一系列的卷积核（或滤波器）扫描输入数据，计算卷积核与输入数据局部区域的点积，从而提取特征。这些卷积核的参数在训练过程中学习得到，能够捕捉到输入数据中的重要特征。
- 池化层（Pooling Layer）：池化层用于降低特征图的空间尺寸，减少参数数量和计算量，同时保持特征的不变性。常见的池化操作有最大池化（Max Pooling）和平均池化（Average Pooling）。
- 全连接层（Fully Connected Layer）：在多个卷积和池化层之后，全连接层将学习到的特征映射到样本的标签空间，用于分类或回归任务。
- 丢弃层（Dropout Layer）：丢弃层在训练过程中随机丢弃一部分神经元的激活值，以防止过拟合，增强模型的泛化能力。

常用的深度学习框架

- 简易和快速的原型设计
- 自动梯度计算
- 无缝CPU和GPU切换

PaddlePaddle

Microsoft CNTK

TensorFlow

Chainer

Caffe2

Keras

theano

PYTORCH

APACHE mxnet

Tensorflow安装步骤

- 1.管理员方式打开命令行，然后运行：
  - pip install wrapt --upgrade --ignore-installed

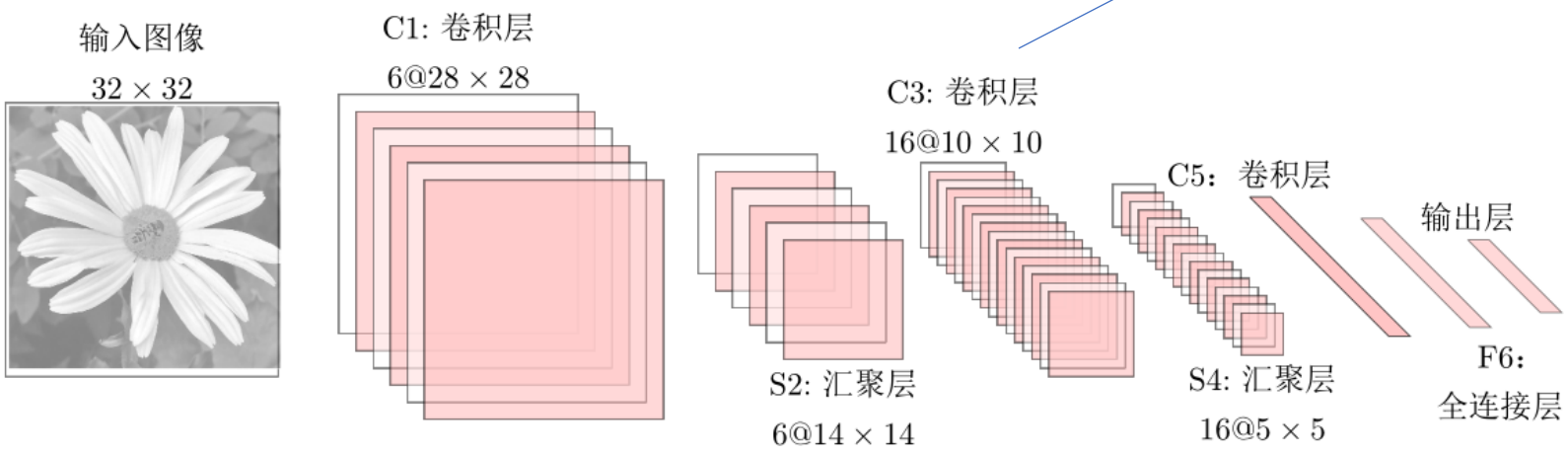
- 2. 然后再运行
  - pip install tensorflow-cpu==2.12.0 -i https://pypi.tuna.tsinghua.edu.cn/simple --user
  - 如果没有出现错误说明安装成功
  - （安装的过程中所出现的错误或警告信息全部可以忽略）

  - pip install numpy==1.22.0 -i https://pypi.tuna.tsinghua.edu.cn/simple --user

LeNet-5

- LeNet-5 是一个非常成功的神经网络模型。
  - 基于 LeNet-5 的手写数字识别系统在 90 年代被美国很多银行使用，用来识别支票上面的手写数字。
  - LeNet-5 共有 7 层。

需要多少个卷积核?



LeNet-5

- 1、输入层（Input layer）
  - 输入层接收大小为 的手写数字图像，其中包括灰度值（0-255）。在实际应用中，我们通常会对输入图像进行预处理，例如对像素值进行归一化，以加快训练速度和提高模型的准确性。
- 2、卷积层C1（Convolutional layer C1）
  - 卷积层C1包括6个卷积核，每个卷积核的大小为5x5，步长为1，填充为0。因此，每个卷积核会产生一个大小为28x28的特征图（输出通道数为6）。
- 3、采样层S2（Subsampling layer S2）
  - 采样层S2采用最大池化（max-pooling）操作，每个窗口的大小为2x2，步长为2。因此，每个池化操作会从4个相邻的特征图中选择最大值，产生一个大小为14x14的特征图（输出通道数为6）。

LeNet-5

- 4、卷积层C3（Convolutional layer C3）
  - 卷积层C3包括16个卷积核，每个卷积核的大小为5x5，步长为1，填充为0。因此，每个卷积核会产生一个大小为10x10的特征图（输出通道数为16）。

- 5、采样层S4（Subsampling layer S4）
  - 采样层S4采用最大池化操作，每个窗口的大小为2x2，步长为2。因此，每个池化操作会从4个相邻的特征图中选择最大值，产生一个大小为5x5的特征图（输出通道数为16）。

LeNet-5

- 6、全连接层C5（Fully connected layer C5）
  - C5将每个大小为5x5的特征图拉成一个长度为400的向量，并通过一个带有120个神经元的全连接层进行连接。120是由LeNet-5的设计者根据实验得到的最佳值。
- 7、全连接层F6（Fully connected layer F6）
  - 全连接层F6将120个神经元连接到84个神经元。
- 8、输出层（Output layer）
  - 输出层由10个神经元组成，每个神经元对应0-9中的一个数字，并输出最终的分类结果。在训练过程中，使用交叉熵损失函数计算输出层的误差，并通过反向传播算法更新卷积核和全连接层的权重参数。

LeNet-5

C1层（第1个卷积层）: 有6个5x5的卷积核，6 * (5x5 + 1) = 156个参数。

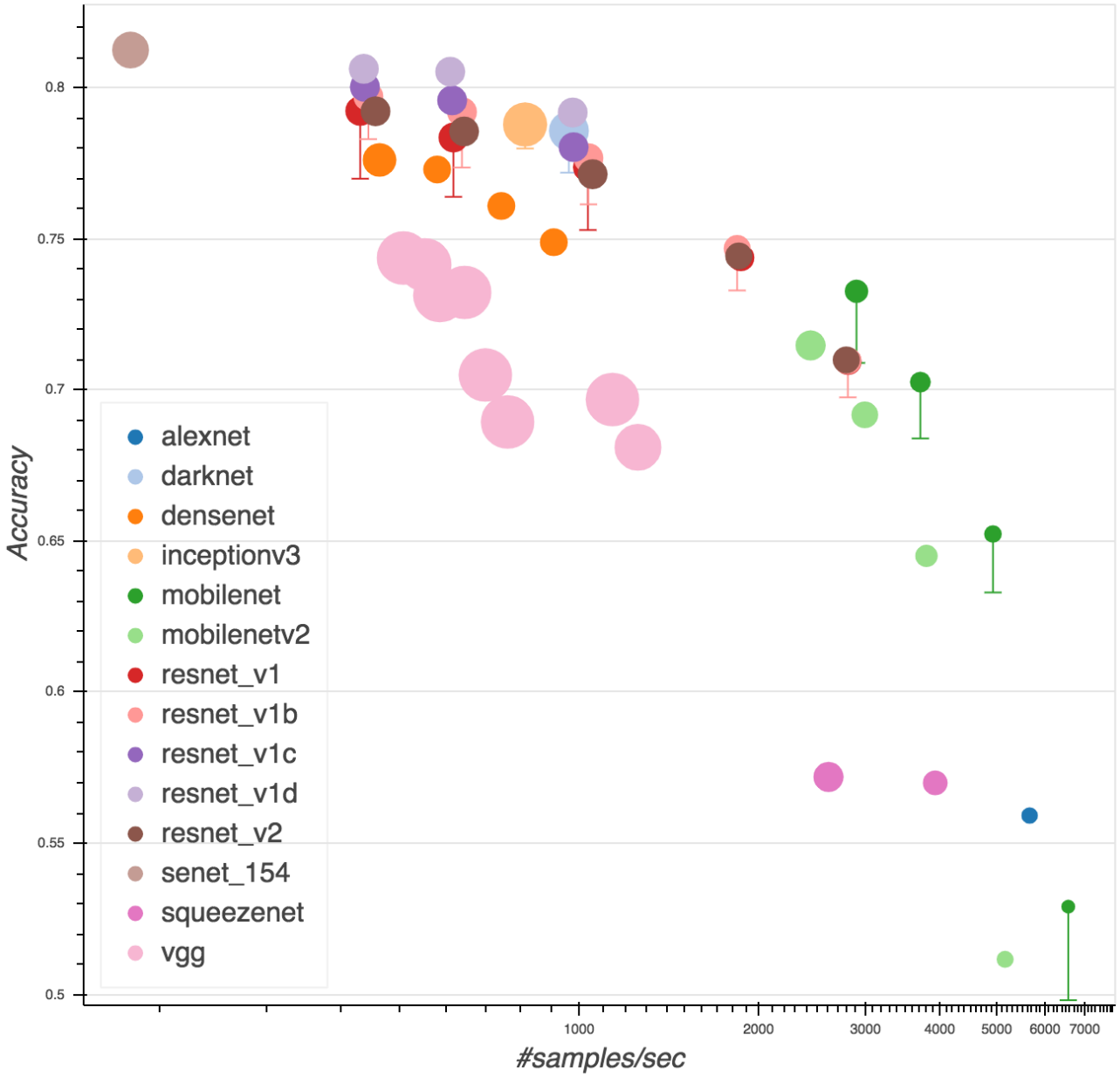
C3层（第2个卷积层）: 有16个5x5的卷积核，16 * (5x5 + 1) = 320个参数。

C5层（第3个卷积层）: 有120个5x5的卷积核，共有120 * (5x5 * 16 + 1) = 48120个参数。

F6层（第1个全连接层）: 输出120个特征连接到84个神经元，有84 * (120 + 1) = 10164个参数。

Output层（第2个全连接层）: 输出84个特征连接到10个神经元，有10 * (84 + 1) = 850个参数。

一共有156 + 320 + 48120 + 10164 + 850 = 59,590个参数

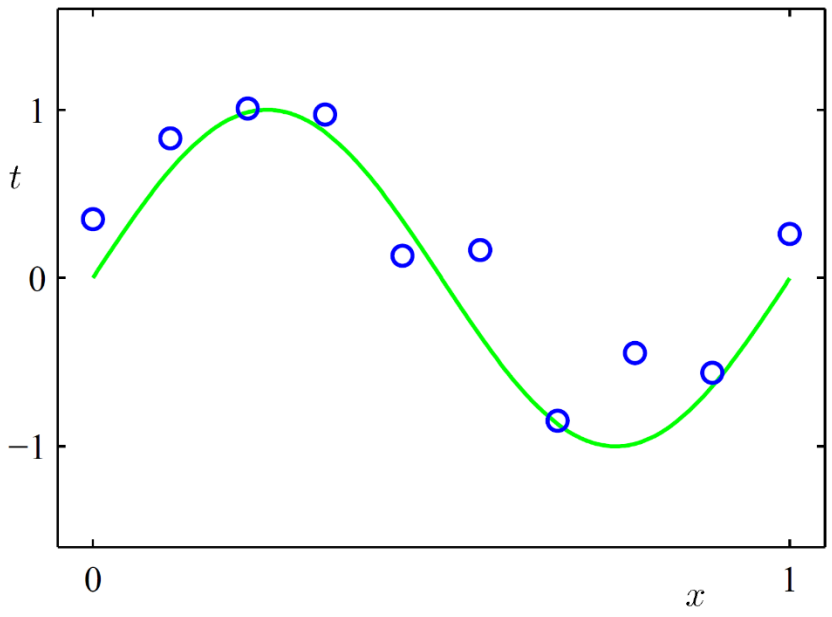


GluonCV 模型“动物园”

https://gluon-cv.mxnet.io/model_zoo/classification.html

过拟合的处理

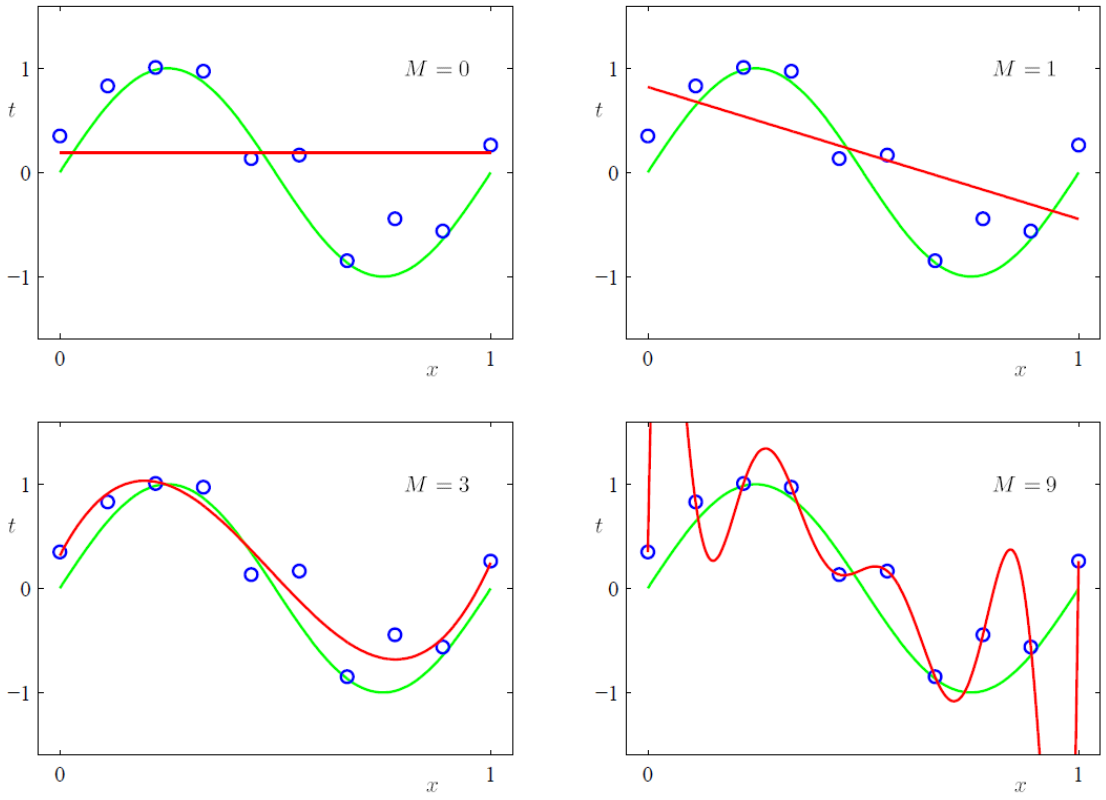
示例：多项式回归



模型 $$y(x, \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M$$

损失函数 $$E(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \boldsymbol{w}) - t_n\}^2$$

Which Degree of Polynomial?



A model selection problem

M = 9 → E(w) = 0: This is overfitting

泛化错误

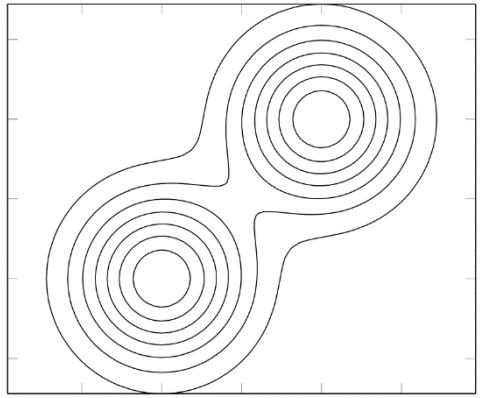
期望风险

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x},y)\sim p(\mathbf{x},y)}[\mathcal{L}(f(\mathbf{x}), y)],$$

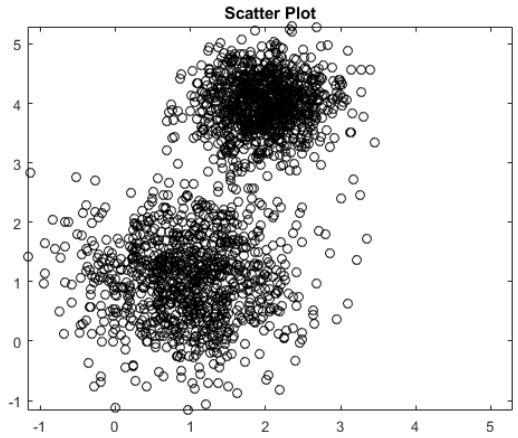
经验风险

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

真实分布 p_r



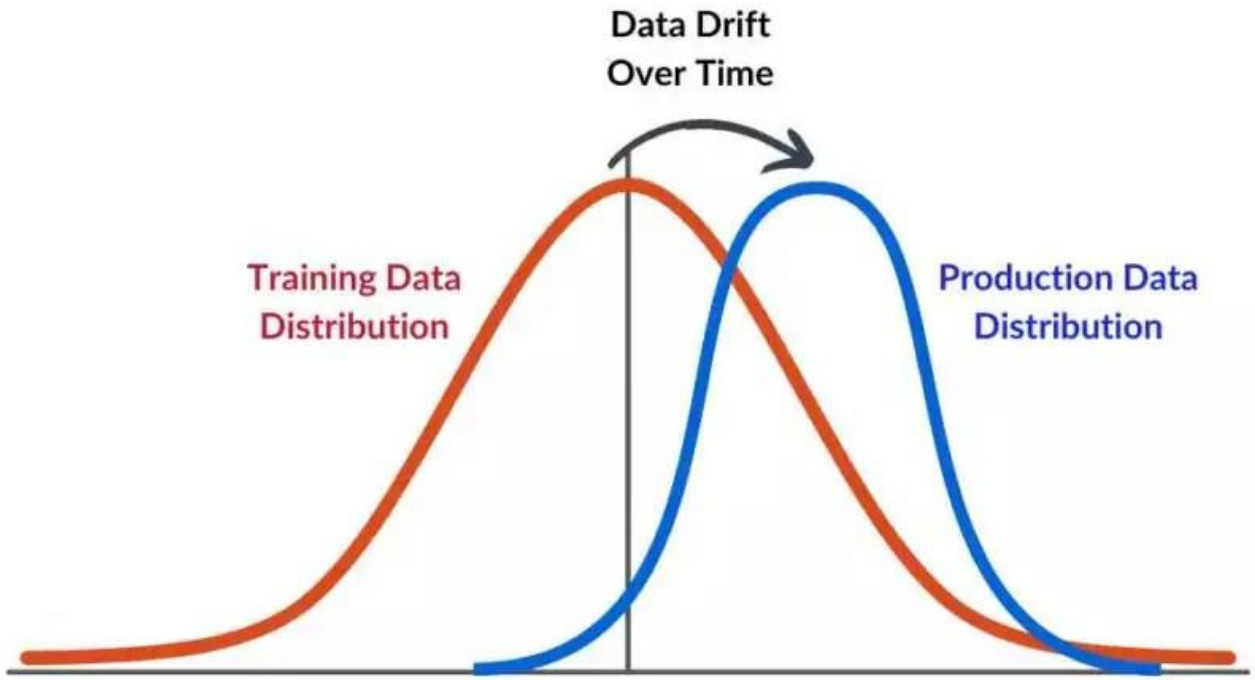
$\neq$



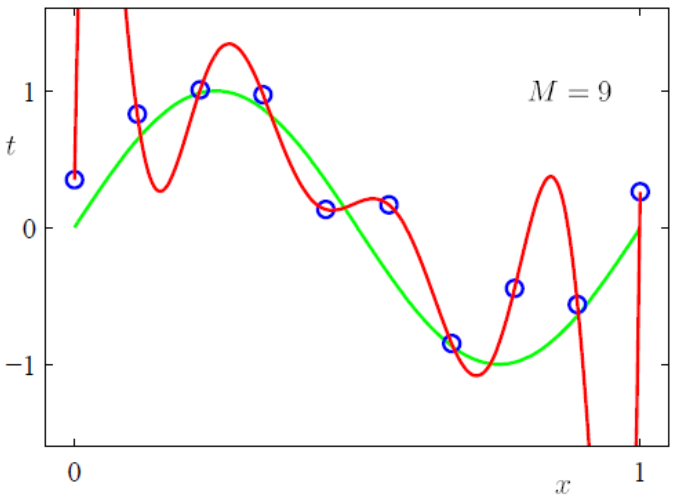
$$\mathcal{G}_{\mathcal{D}}(f) = \mathcal{R}(f) - \mathcal{R}_{\mathcal{D}}^{emp}(f)$$

泛化错误

数据漂移


Data Drift
Over Time
Training Data
Distribution
Production Data
Distribution

防止过拟合: 正则项



	$M = 0$	$M = 1$	$M = 3$	$M = 9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

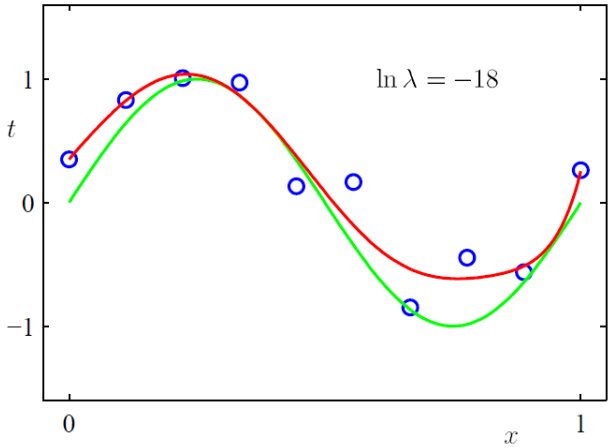
As order of polynomial M increases, so do coefficient magnitudes!

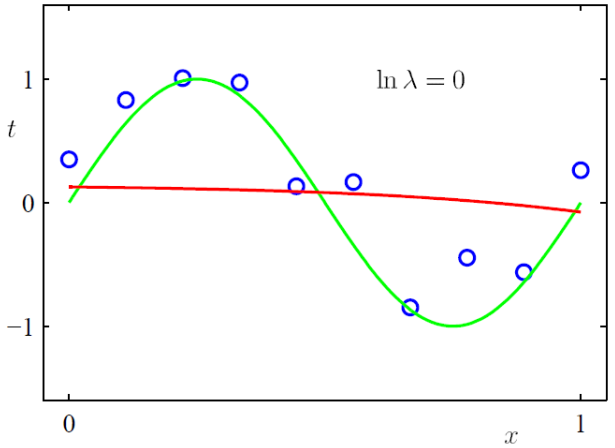
$$\tilde{E}(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \boldsymbol{w}) - t_n\}^2 + \frac{\lambda}{2}||\boldsymbol{w}||^2$$

对大的系数进行惩罚

防止过拟合: 正则项

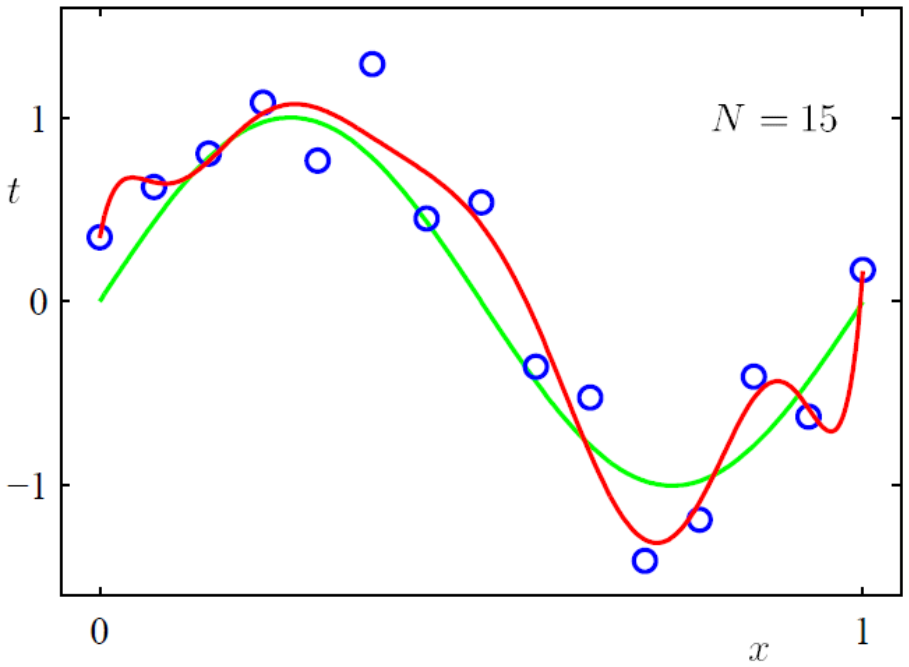
$$\tilde{E}(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \boldsymbol{w}) - t_n\}^2 + \frac{\lambda}{2}||\boldsymbol{w}||^2$$

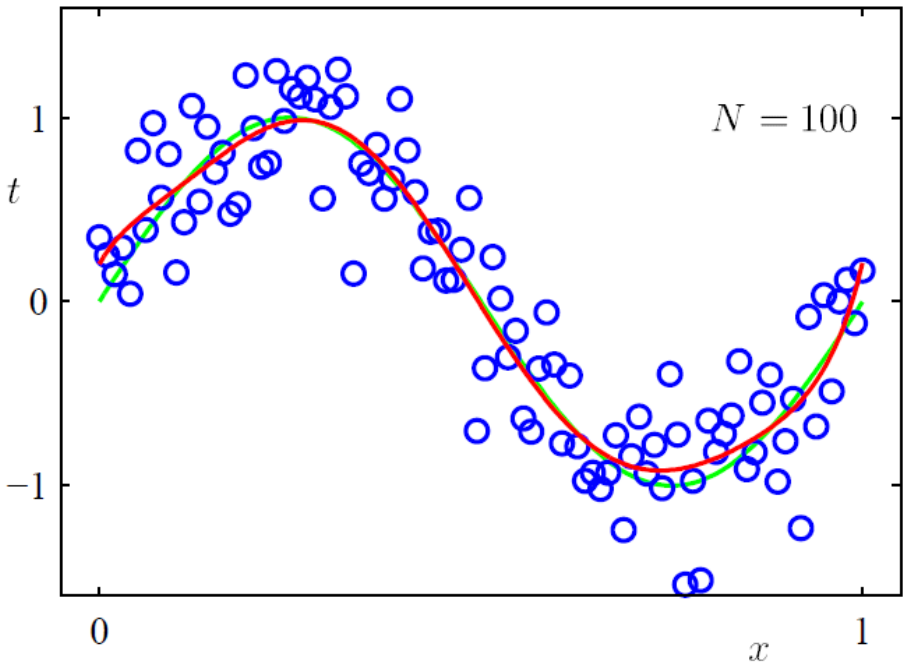




	$\ln\lambda = -\infty$	$\ln\lambda = -18$	$\ln\lambda = 0$
$w_0^\star$	0.35	0.35	0.13
$w_1^\star$	232.37	4.74	-0.05
$w_2^\star$	-5321.83	-0.77	-0.06
$w_3^\star$	48568.31	-31.97	-0.05
$w_4^\star$	-231639.30	-3.89	-0.03
$w_5^\star$	640042.26	55.28	-0.02
$w_6^\star$	-1061800.52	41.32	-0.01
$w_7^\star$	1042400.18	-45.95	-0.00
$w_8^\star$	-557682.99	-91.53	0.00
$w_9^\star$	125201.43	72.68	0.01

防止过拟合: 增大数据集

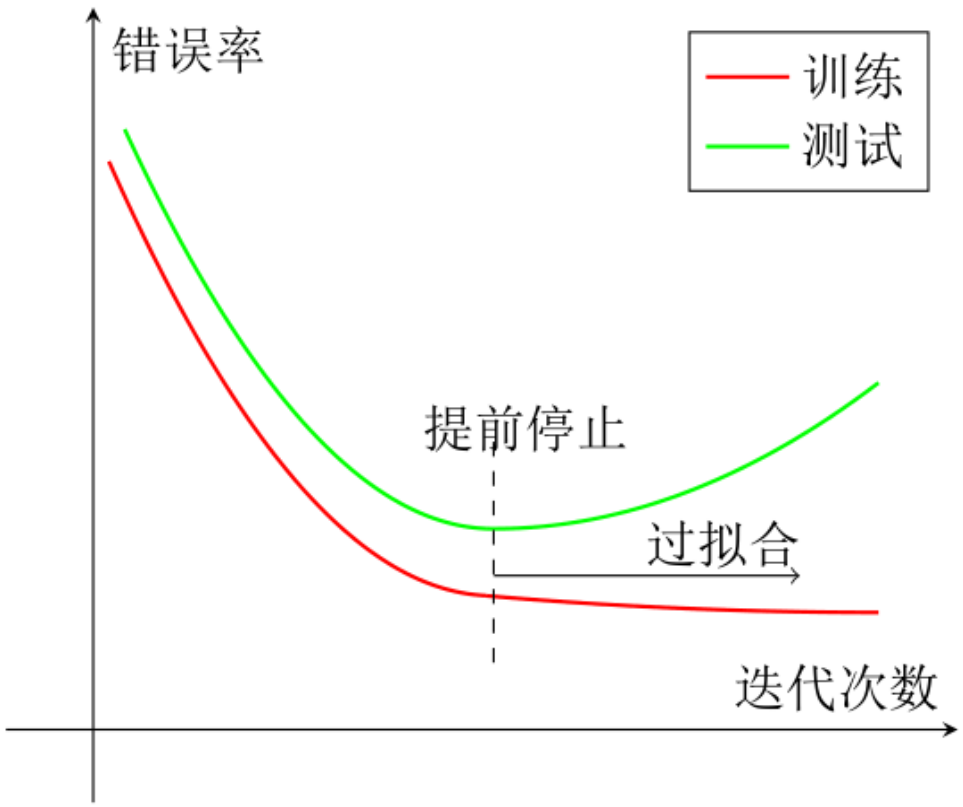




数据增强（Data Augmentation）

- 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。
  - 图像数据的增强方法:
    - 旋转（Rotation）：将图像按顺时针或逆时针方向随机旋转一定角度;
    - 翻转（Flip）：将图像沿水平或垂直方法随机翻转一定角度;
    - 缩放（Zoom In/Out）：将图像放大或缩小一定比例;
    - 平移（Shift）：将图像沿水平或垂直方法平移一定步长;
    - 加噪声（Noise）：加入随机噪声。

防止过拟合: 干扰优化过程

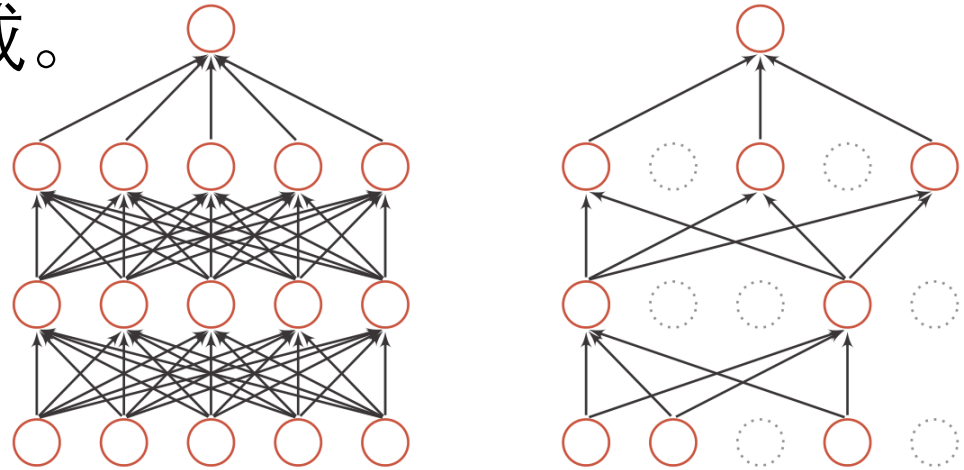


防止过拟合: 丢弃法（Dropout Method）

- 对于一个神经层$y = f(Wx + b)$，引入一个丢弃函数$d(\cdot)$使得$y = f(Wd(x) + b)$。

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$

- 其中$m \in \{0,1\}^d$ 是丢弃掩码（dropout mask），通过以概率为p的贝努力分布随机生成。



(a) 标准网络　　(b) Dropout 后的网络